

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**МНОГОПЛАТФОРМЕН ГРАФИЧЕН
ИНТЕРФЕЙС: ПРОЕКТИРАНЕ И
ЕКСПЕРИМЕНТАЛНА ОЦЕНКА НА
НЕЗАВИСИМОСТТА ОТ ПЛАТФОРМЕННИЯ
ИНСТРУМЕНТАРИУМ**

ДИПЛОМНА РАБОТА

за придобиване на ОКС „Магистър“

Изготвил:

Алекс Цветанов, фак. № **[TODO: фак. номер (магистратура)]**

Специалност: **[TODO: специалност]**

Научен ръководител:

[TODO: звание, име на ръководителя]

София, 2026

**[TODO: Тук се подвързва УТВЪРДЕНОТО ДИПЛОМНО ЗАДАНИЕ (подписано, с
печат)]**

[TODO: Тук се подвързва ДЕКЛАРАЦИЯТА ЗА АВТОРСТВО]

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
1.1. Подходи за разработка на многоплатформени графични интерфейси	3
1.2. Съществуващи многоплатформени фреймуорци	4
1.3. Архитектурата на .NET MAUI	6
1.4. Изводи от литературния преглед	8
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	9
2.1. Критерии за оценка	9
2.2. Избор на целеви език и на езиковия стандарт	10
2.3. Стратегия за изграждане	13
2.4. Модел на собствеността върху обектите	14
2.5. Избор на платформени слоеве и на реда им	16
2.6. Подход към декларативното описание	16
2.7. Обобщение на направения избор	17
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	19
3.1. Синтез на общата структура	19
3.2. Проектиране на слоя за графични примитиви	21
3.3. Договори на виртуалните изгледи	21
3.4. Проектиране на слоя за преобразуване	22
3.5. Проектиране на алгоритмите за разположение	24
3.6. Проектиране на системата от свойства	26
3.7. Проектиране на потребителския програмен интерфейс	27
3.8. Формат на декларативното описание	28
3.9. Проектиране на платформените слоеве	28
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	30
4.1. Обща структура на изходния код	30
4.2. Организация на компилацията	30
4.3. Реализация на слоя за преобразуване	31

4.4. Реализация на алгоритмите за разположение	33
4.5. Реализация на системата от свойства	33
4.6. Реализация на платформените слоеве	34
4.7. Реализация на декларативния слой	36
4.8. Обработка на събития от интерфейса	38
4.9. Дисциплина на разработка и осигуряване на верността	39
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА НА ЕКВИВАЛЕНТНОСТТА	42
5.1. Постановка на експеримента	42
5.2. Автоматизирано заснемане	43
5.3. Правила за оценяване	45
5.4. Количествено оценяване	47
5.5. Достоверност на измерването	50
5.6. Предварителна регистрация на хипотезите	51
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	53
6.1. Степен на съвпадение	53
6.2. Представителни съпоставки	55
6.3. Систематизация на нарушенията на независимостта	60
6.4. Цена на реализацията	65
6.5. Ограничения на изследването	70
ЗАКЛЮЧЕНИЕ	72
ЛИТЕРАТУРА	76
ПРИЛОЖЕНИЯ	78
Приложение А. Пълен изходен текст на срез Button	78
Приложение Б. Публичен програмен интерфейс на системата	81
Приложение В. Съвпадение по интерфейси	82
Приложение Г. Екранни форми	83

УВОД

Разработката на графичен потребителски интерфейс за няколко операционни системи едновременно поставя задача, чието обичайно решение се приема без проверка: всеки от съществуващите многоплатформени инструменти твърди, че едно описание на интерфейса стига за еднакво поведение върху различни платформи. Проверката на това твърдение е трудна, защото за разлика от изчислителния код, чиято коректност се доказва с тест, „правилният“ вид на потребителски интерфейс не е записан никъде във вид, който машина може да сравни. Това е частен случай на *проблема за оракула* — невъзможността да се отличи желаното от нежеланото поведение при даден вход — разгледан систематично в [1].

Настоящата работа изхожда от наблюдението, че проверката не е принципно невъзможна, а само лишена от еталон. Ако съществува зряла реализация, приета за коректна, тя може да служи не като образец за подражание, а като *измервателен инструмент* — източник на еталонни изображения, спрямо които независима реализация на същото описание се сравнява количествено. Смяната на ролята е принципна: съпоставяната реализация престава да бъде „копие“ и става второ, независимо решение на една и съща задача, а разликите между двете стават измеримо свойство на абстракцията, а не въпрос на мнение. Така изследването се отличава от инженерен отчет за пренасяне на код между езици: то проверява хипотеза, която без независим втори изпълнител на едно и също описание би останала недостъпна за проверка.

Целта на дипломната работа е да установи доколко наблюдаваното поведение на графичен потребителски интерфейс може да бъде описано независимо от платформения инструментариум, който го изобразява, и къде точно тази независимост се нарушава — чрез работеща реализация на такова описание върху няколко разнородни native инструментариума, методика за количествена проверка на еквивалентността на изобразяването и емпирична оценка на цената на избраната стратегия.

За постигането ѝ са формулирани следните **задачи**:

1. литературен преглед на подходите за многоплатформено описание на интерфейс и анализ на архитектурата, върху която стъпва изследването;
2. анализ на проектните алтернативи — целеви език, стратегия за изграждане, модел на собствеността, набор от платформени слоеве — и обосновка на избора;

3. проектиране на слоева архитектура, в която описанието на интерфейса, изчисляването на разположението и системата от свойства са преносими, а платформено обусловената част е сведена до таблично зададено съответствие;
4. програмна реализация на архитектурата върху четири native инструментариума и един слой без екран;
5. методика и инструментариум за автоматизирано измерване на еквивалентността на изобразяването между две независими реализации на едно и също описание;
6. експериментална проверка върху пълния набор от интерфейси, систематизация на установените разминавания и оценка на цената на реализацията.

Обхватът на работата е логическият слой, който описва интерфейса, изчислява разположението и управлява свойствата, заедно с платформените слоеве, необходими за неговото изобразяване върху избраните native инструментариума. Извън обхвата остават съвместимостните слоеве към предходни поколения на еталонната реализация, вътрешните ѝ помощни пространства от имена и допълненията, които не са част от ядрото ѝ.

Методологична бележка. Хипотезите за цената на реализацията — размер на артефакта, потребена памет, време за реакция — са *регистрирани предварително*, преди да съществува измерване, заедно с изискванията за валидност, на които всяко измерване трябва да отговаря, за да остане погрешна прогноза видима и отчетена като такава, а не реконструирана след факта. Регистрацията е приложена към работата.

Структура на изложението. Раздел I разглежда предметната област и архитектурния шаблон, върху който стъпва изследването. Раздел II излага разгледаните проектни алтернативи и обосновава избора между тях. Раздел III представя проектирането на системата, а Раздел IV — нейната програмна реализация. Раздел V описва методиката за измерване на еквивалентността — самостоятелен принос на работата и предпоставка за валидността на всичко, което следва. Раздел VI представя резултатите — систематизацията на установените нарушения на независимостта и измерената цена на реализацията. Заключението обобщава приносите, ограниченията и насоките за по-нататъшна работа.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

Разделът е подреден от общото към частното: класификация на подходите по вида краен рендиращ механизъм, преглед на конкретните им представители, и накрая — по-подробно — архитектурата на .NET MAUI, която тук изпълнява ролята на еталонен измервателен инструмент, а не на образец за копиране. Тя е разгледана подробно, защото границата, която въвежда, е и предметът на измерването в Раздел VI.

1.1. Подходи за разработка на многоплатформени графични интерфейси

Съществуващите решения се делят на три семейства според това *какво* в крайна сметка рисува пиксел на екрана. Разделението определя всичко останало — вида на крайния контрол, размера на артефакта, достъпа до платформените услуги и, което е предметът на настоящата работа, *дали изобщо съществува понятие „правилен вид“*, спрямо което решението да бъде проверено.

1.1.1. Абстракция над native контроли

Едно описание се преобразува към контролите на *самата* платформа: списъкът е native списък, бутонът е native бутон. Предимствата идват от само себе си — външният вид, поведението при въвеждане, достъпността и езиковите настройки идват от платформата и се променят заедно с нея.

Цената е, че абстракцията трябва да *съгласува* инструментариуми, различни не само по имена, а по модел: различни правила за разположение, различни жизнени цикли, различни конвенции за координати. Именно тук е въпросът, който настоящата работа измерва — доколко такова съгласуване е постижимо и къде се проваля.

1.1.2. Собствен растеризатор

Фреймуоркът не използва native контроли, а рисува всеки елемент сам, върху графично платно. Резултатът е еднакъв на всички платформи *по замисъл*, което премахва целия клас проблеми на съгласуването.

Замяната обаче е реална: интерфейсът престава да бъде native. Той не наследява

автоматично промените във външния вид на платформата, а достъпността, въвеждането с клавиатура и текстовите услуги трябва да бъдат възпроизведени във фреймуорка, който носи и самия растеризатор.

1.1.3. Вграден уеб изглед

Интерфейсът е уеб документ в native обвивка. Този подход има най-ниска цена на входа и най-голяма отдалеченост от платформата; той е и подходът с най-обилно изследвана цена в литературата [2].

1.1.4. Критерии за съпоставяне

Таблица 1. Съпоставяне на трите подхода за многоплатформен графичен интерфейс

Критерий	Над native контроли	Собствен растеризатор	Вграден уеб изглед
краен контрол	native	собствен	документен елемент
външен вид след промяна в платформата	следва я сам	изисква работа	изисква работа
достъпност, въвеждане	от платформата	във фреймуорка	от уеб слоя
размер на артефакта	малък	+ растеризатор	+ уеб механизъм
среда за изпълнение	по избор	обичайно да	да
наличие на еталон	да	не (сам си е еталон)	частично

Последният ред от (Табл. 1) е решаващ за настоящата работа. При собствен растеризатор въпросът „съответства ли изобразеното“ не стои — изобразеното *по определение* е правилното, защото няма втора инстанция, спрямо която да се сравнява. Само при абстракция над native контроли твърдението за съответствие е смислено, проверимо — и, както показва §1.4, непроверено.

1.2. Съществуващи многоплатформени фреймуорци

1.2.1. Xamarin.Forms и .NET MAUI

Историческата линия е поучителна, защото съдържа точно един архитектурен завои. Xamarin.Forms свързва всяка контрола с платформената чрез *renderer* — клас,

наследяван по йерархия, който съчетава създаването на *native* изгледа с пренасянето на всяко свойство. Наследяването означава, че разширяването на една контрола изисква наследяване на нейния *renderer*, обвързан с платформата.

.NET MAUI [3, 4] заменя този механизъм с *handler*: обектът вече не се наследява, а носи *таблица* от съответствия „свойство → действие“. Разликата е между разширяване чрез наследяване и разширяване чрез вписване в таблица — второто е съставимо, не изисква познаване на йерархията и позволява една контрола да бъде доразвита, без нищо да се наследява. Именно този шаблон е предметът на §1.3 и основата, върху която стъпва настоящата работа.

1.2.2. Flutter

Представител на семейството със собствен растеризатор [5]. Изгражда интерфейса от собствени примитиви върху графично платно, което дава пълна еднаквост между платформите и пълна свобода в анимациите, но и означава, че всяка контрола, всяко поведение при въвеждане и всяка услуга за достъпност съществуват във фреймуорка, а не в платформата.

По критериите от §1.1 Flutter заема крайното положение на второто семейство: пълна еднаквост между платформите и артефакт, който носи целия слой за изобразяване. За настоящото изследване той е показателен по обратен начин — при него описаният тук метод за измерване е *неприложим*, защото липсва втора независима реализация на същото описание, спрямо която да се сравнява.

1.2.3. Qt

Най-старият зрял C++ фреймуорк в областта и единственият пряк съсед на настоящата работа по език [6]. Съдържа и двата подхода: по-старият слой със *widgets* рисува сам, като подражава на външния вид на платформата, а по-новият декларативен слой е самостоятелен растеризатор със собствен език за описание.

Разглеждането му е особено уместно, защото Qt решава — на същия език — и задачата за собствеността без събирач на боклук: дървото от обекти е с притежаване от родител към дете, а наблюдаващите препратки са отделен тип. Настоящата работа стига до сходно разпределение на ролите по независим път (§2.4), което е довод, че решението е принудено от задачата, а не въпрос на предпочитание.

1.2.4. Avalonia, Slint и React Native

Avalonia [7] е собствен растеризатор върху управлявана среда, с декларативен език, силно повлиян от предходно поколение настолни технологии; позиционира се като преносим наследник на тези технологии, което го поставя във второто семейство въпреки близостта на езика му за описание до първото.

Slint също рисува сам, но с вграден собствен език за описание и с изричен прицел към устройства с ограничени ресурси — което го прави най-близкия по цели представител на второто семейство за вградени приложения.

React Native [8] принадлежи на първото семейство: изгражда native контроли, но описанието се изпълнява в скриптов език, а границата между двете страни е асинхронна. Точно тази граница е предметът на измерването в [2] — изследване, което мери *цената* на връзката, но не и съответствието на изобразеното.

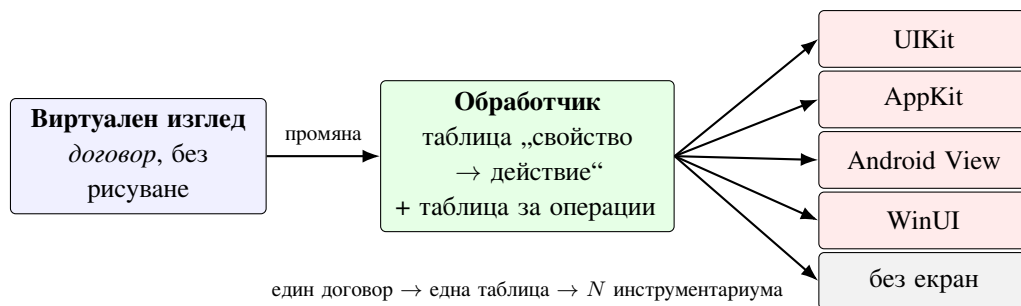
1.3. Архитектурата на .NET MAUI

Централната идея, върху която стъпва цялата работа, е че разглежданата реализация **не е растеризатор**. Тя не рисува нито един пиксел сама. Тя е абстракция, която преобразува едно многоплатформено описание на интерфейса към няколко native инструментариума, и е организирана около един-единствен повтарящ се шаблон, приложен еднакво за всяка контрола.

1.3.1. Шаблонът Handler: виртуален изглед, mapper, native контрола

Връзката е тристранна и е показана на (Фиг. 1). Най-простият ѝ срез — бутон — се състои от: *договор* (интерфейс само със свойства, без изобразяване); *обработчик* [9], който носи две таблици — за свойства и за императивни операции; и *native контрола* на всяка платформа [10, 11, 12]. Таблицата за свойства съпоставя име на свойство с функция, която прилага текущата му стойност; таблицата за операции прави същото за действия, които нямат стойност (например „получи фокус“).

Границата минава точно тук: договорот и таблиците са преносими, а само реализациите на отделните действия са платформени. Затова добавянето на инструментариум е *добавяне* на реализации, а не изменение на съществуващи — свойство, което Раздел III запазва съзнателно.



Фигура 1. Шаблонът на преобразуването. Виртуалният изглед носи само свойства; обработчикът съпоставя всяко свойство на действие, което го прилага върху native контролата. Смяната на инструментариума заменя само дясната колона.

1.3.2. Слоеста организация и ред на зависимостите

Зависимостите между слоевете вървят в една посока — графични примитиви, договори, обработчици, платформени слоеве, разположение, контроли, декларативен слой, услуги на устройството, инициализация — и този ред е задължителен и при независимо изграждане на такъв слой. Причината не е организационна: слой, който може да достигне до платформения, престава да бъде преносим, дори ако в момента не го достига. Пълното разглеждане на структурата, вече в реализирания вид, е в §3.1.

1.3.3. Системата от свойства и свързването на данни

Системата от свойства е вторият механизъм, който се повтаря навсякъде. Всяко свойство е описано от споделен описател (име, стойност по подразбиране, обратни извиквания), а всяка инстанция държи само стойността. Върху това стъпват три неща: известяване при промяна — което задвижва и таблиците от §1.3; *привеждане* (coercion) на стойността към допустим обхват; и *предимство* между едновременно валидни източници на една и съща стойност — подразбираща се, от стил, от свързване, от ръчно присвояване и т.н.

Предимството е частта, която най-лесно се подценява. То не е вътрешна подробност, а наблюдаемо поведение: определя дали изрично зададен от приложението цвят надделява над цвят от стил, и дали „незададено“ се различава от „зададено на стойността по подразбиране“. Раздел VI показва, че точно това разграничение е източник на цял клас разминавания.

1.4. Изводи от литературния преглед

Прегледът води до четири констатации, всяка от които се връзва със задача от Увода.

Първо. Твърдението за еднакво поведение върху различни платформи е *общо за цялото семейство* решения, които стъпват на native контроли, и никъде не е придружено от количествена проверка. Съпоставителните изследвания измерват друго — режийни разходи на границата към платформата [2], потребление на ресурси, удобство за разработчика — но не и степента, в която двете реализации на едно описание изобразяват едно и също.

Второ. Причината за тази празнина не е незаинтересованост, а липса на *оракул*: за разлика от изчислителния код, „правилният“ вид на интерфейс не е записан във вид, годен за машинно сравнение [1]. Изследванията върху визуално тестване на интерфейс [13] заобикалят това, като сравняват система със *самата нея* в предишен момент — което открива регресии, но не може да провери твърдение за съответствие между две различни реализации.

Трето. Съществува обаче положение, при което оракул е наличен: когато една зряла реализация се приеме за меродавна и втора, независима реализация изпълнява *същото* описание. Тогава разликата между двете изображения е измерима величина. Това е наблюдението, върху което стъпва настоящата работа.

Четвърто. Архитектурата с таблично преобразуване (§1.3) е особено подходяща за такава проверка, защото границата между преносимото и платформеното е *един* и е явен. Реализация, която го запазва, позволява разминаванията да бъдат отнасяни към определен слой; реализация, която го събаря в един растеризатор, не позволява това.

Тези четири констатации определят и подредбата на останалото изложение: избор на средства и стратегия (Раздел II), запазване на тази граница при проектиране и реализация (Раздели III и IV), изграждане на оракула в работеща методика (Раздел V) и получените от нея резултати (Раздел VI).

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

Всяко от разгледаните по-долу решения е било действителна възможност в момента на вземането му, а не реторична алтернатива, въведена за да бъде отхвърлена. Разделът излага първо критериите, после вариантите по петте определящи решения, и накрая — обобщена оценка. Където решението почива на измерване, а не на преценка, измерването е посочено.

2.1. Критерии за оценка

За да не бъде изборът въпрос на вкус, вариантите се съпоставят по пет предварително изброени критерия:

- К1. **Проверимост** — може ли резултатът от решението да бъде проверен автоматизирано, срещу външен източник на истина, без човешка преценка.
- К2. **Архитектурна вярност** — запазва ли решението разделението, което прави описанието преносимо, или го размива.
- К3. **Независимост от среда за изпълнение** — изисква ли решението допълнителна среда, вградена в крайния артефакт.
- К4. **Безопасност по конструкция** — превръща ли решението цял клас грешки в невъзможни, вместо да разчита на дисциплина.
- К5. **Цена при добавяне на платформа** — какво трябва да се напише отново при всеки нов платформен слой.

Критерий К1 е поставен пръв съзнателно. Работата изследва твърдение за еквивалентност; решение, което подобрява който и да е друг критерий за сметка на проверимостта, обезсмисля изследването.

2.2. Избор на целеви език и на езиковия стандарт

Таблица 2. Разгледани целеви езици по критериите K1–K5

Критерий	C++26	Rust	Zig	C + обвивки
K1 проверимост	пълна (пренасяне на тестовите е пряко)	пълна	пълна	пълна
K2 архитектурна вярност	висока	висока	висока	ниска (без обобщени типове)
K3 независимост от среда	да	да	да	да
K4 безопасност по конструкция	чрез типовата система	<i>най-висока</i>	средна	ниска
K5 цена за платформа	<i>най-ниска</i> (пряк достъп)	висока (обвързване)	висока	средна

Избран е **C++26** (Табл. 2). Rust печели по K4 — проверката на заемането прави безопасността свойство на езика, не на дисциплината — но губи по K5, който тук тежи повече: три от петте целеви инструментариума се обръщат от C++ без междинен слой за маршалиране, а за Apple платформите смесеният език дава на една единица за превод достъп едновременно до C++ и native обектния модел — native обектите се държат пряко в C++ клас, не се пренасят през граница. При Rust всеки от петте слоя би започнал с изграждане на обвързване; а тъй като K4 се оказва постижим и в C++ чрез само преместваеми типове (§2.4), изборът пада върху C++26.

Изборът на *конкретния стандарт* обаче не е формалност: задачата изисква (а) работа от изпълнение, преместена в превода; (б) заместител на рефлексията, проверим от компилатора; и (в) модел на собственост, изразим в типовата система. Следващите подпараграфи изброяват средствата, какво заместват и къде ефектът е реално измерим.

2.2.1. Изчисления по време на компилация

Оригиналът изгражда голяма част от структурите си при стартиране — таблици за преобразуване, идентификатори на типове, описатели на свойства — работа, платена при всяко пускане. В C++26 значителна част се измества в превода: `constexpr` и `constexpr` правят изчислението *задължително* компилационно, не „по възможност“

(consteval не компилира при извикване в контекст на изпълнение); if consteval позволява различна реализация в двата режима без два отделни текста. Резултатът е *константно инициализирани* константи и таблици, готови в артефакта, без изграждане при пускане и без неопределения ред на инициализация между единиците за превод.

Делът на началното време, който отпада така, *не* се твърди: измерването по H2b-1 не е проведено (§6.4) — механизмът е описан и предварително регистриран, но без число, разликата между обосновка и реклама.

2.2.2. Вграждане на описанието в превода

Декларативното описание обичайно се чете от файл и се разбира при стартиране. #embed (C++26) прави съдържанието на файла част от единицата за превод, така че разборът може да стане с constexpr код — по време на компилация. Ползата е *в момента, в който грешката се открива*: неправилно описание престава да бъде отказ при стартиране и става грешка при компилация.

Средството не е налично във всички поддържани компилатори; за една платформа това наложи резервен път — масив от байтове чрез отделна стъпка в изграждането (§4.7), със същия резултат срещу цената на допълнителна зависимост.

2.2.3. Концепции вместо проверки по време на изпълнение

Концепциите (concept, requires) заместват два механизма на оригинала: ограниченията върху обобщените типове, декларативни там, но проверявани отчасти при изпълнение; и — по-важно тук — *незадължителните точки за разширение*. Базовият клас на обработчиците открива при компилация дали производният предоставя дадена точка за разширение чрез изискване за наличие на израз, така че производният изброява само наистина предоставеното, а извикването на несъществуваща точка е грешка при компилация, не безшумно нищо.

2.2.4. Типизиран достъп вместо изтриване на типа

Системата от свойства на оригинала стъпва върху общия базов тип на средата за изпълнение: всяка стойност се *опакова*, а извличането проверява типа при изпълнение. Тук описателят на свойство е шаблон по типа на стойността — без опаковане, без информация за типа при изпълнение, без заделяне за стойност, побираща се в самия обект. Предимството между източниците на стойност е опаковано в едно 64-битово число,

чийто целочислен ред *съвпада* с реда на предимството (§3.6): разрешаването се свежда до едно целочислено сравнение.

2.2.5. Наблюдения без копиране

`std::span` и `std::string_view` предават последователности и низове без заделяне и копиране. В слой, в който имена на свойства се сравняват при всяко преминаване през таблицата, разликата между сравнение на изглед и сравнение на собствен низ не е козметична — второто заделя памет на горещия път, изпълняван при всяка промяна.

2.2.6. Детерминистично освобождаване

Липсата на автоматично събиране на боклука е ограничение, но носи свойство, което управляваната среда няма: *моментът* на освобождаване се определя от обхвата, не от събирач — разрушителят освобождава native ресурса веднага.

Тук е нужна съдържаност: предварително регистрираната хипотеза **H2b-3** твърди липса на разлика при установен режим на изобразяване (§5.6), защото при малка купчина събирачът рядко надхвърля бюджета за кадър, а времето за кадър се изразходва предимно в native инструментариума — еднакъв и в двата случая. Очакваният измерим ефект е в следата на паметта (H2a) и в началното време (H2b-1), не в честотата на кадрите.

2.2.7. Само преместваеми типове

Само преместваемите типове превръщат модела на собствеността от съглашение в проверимо от компилатора свойство (§2.4). Изтритият копиращ конструктор е старо средство; новото е последователната поддръжка в стандартната библиотека, включително за съхранение на извиквани обекти, което позволява един обработчик да улавя само преместваемо състояние.

2.2.8. Обобщение: къде езикът действително дава ефект

Средство	Какво замества	Очакван измерим ефект
<code>constexpr/constexpr</code>	изграждане при стартиране	начално време, размер
<code>#embed</code>	четене и разбор при стартиране	начално време; момент на грешката
концепции	рефлексия, проверки при изпълнение	момент на грешката (не бързодействие)
типизиран описател	опаковане на стойности	следа на паметта, заделяния
опаковано предимство	сравнение на обекти	време при промяна на свойство
<code>span/string_view</code>	копия на низове	заделяния по горещия път
детерминистично разрушаване	събиране на боклука	следа на паметта
само преместваеми типове	дисциплина по съглашение	коректност, не бързодействие

Третата колона е нарочно *проверимо твърдение*, не реклама: три от осемте средства не влияят на бързодействието — плащат в коректност или в момента на откриване на грешка. Смесването на двата вида полза е обичайният начин технически избор да бъде подкрепен с числа, които не го подкрепят.

Установено ограничение, непредвидено предварително. Стандартната библиотека на един компилатор не предоставя типа за само-преместваема функция, макар да е част от стандарта; реализацията носи собствена, временна заместваща реализация — показателно, че изборът на стандарт не гарантира наличие на средствата му във всички компилатори, реална цена на решението, не подробности.

2.3. Стратегия за изграждане

Разгледани са три стратегии:

1. **Автоматизирано преобразуване** на изходния текст на еталонната реализация. Отхвърлена: преобразуваният текст носи модела на паметта на изходния език (противоречи на K3 и K4), а резултатът не е проверим по K1 — преобразуването не може да се твърди за коректно, освен чрез проверка, която връща задачата в началото.
2. **Ново прочитане** — архитектура, вдъхновена от еталонната, но изградена наново. Отхвърлена по K1: без съответствие едно към едно между двете реализации няма общо

описание, което и двете да изпълняват, тоест няма контролно условие за сравнение.

3. Послойно изграждане, водено от тестове. Избрана.

Избраната стратегия работи слой по слой отдолу нагоре и за всяка компонента изисква поведението да бъде *прочетено*, не предположено, от три различни източника, всеки отговарящ на различен въпрос:

Въпрос	Източник	Какво дава
Каква е формата?	описание на публичната повърхност	имена, сигнатури, наличност
Как трябва да се държи?	тестовите на еталона	стойности по подразбиране, гранични случаи, ред
Как е реализирано?	изходният текст на еталона	алгоритми, свързване с платформата

Разделянето не е педантизъм: формата не съдържа поведение, тестовите не съдържат алгоритъм, а изходният текст не казва кое от направеното е договор и кое — подробност. Замяната на един източник с друг е обичайният начин да се получи правдоподобна, но невярна реализация.

Редът на разрешаване при противоречие е фиксиран предварително. Между *форма* и *поведение* противоречие практически не възниква — те описват различни неща. Реалният случай е между *изходния текст* на еталона и *действителното му поведение*: изходният текст е снимка от момент, а измерването е срещу разпространената версия. Правилото, записано преди първото приложение, е: **предимство има изобразеното** — изходният текст остава източник за механизма, не за стойност, която оттогава се е променила.

Първо приложение: изходният текст поставя долна граница от 44 единици за размера на превключвател, а разпространената версия изобразява 18, без долна граница (измерено на две платформи). Системата премахна границата, отбелязана като документирано отклонение с цитат на двата източника; резултат — два интерфейса преминаха от разминаване към точно съвпадение.

2.4. Модел на собствеността върху обектите

Това е решението с най-големи последици — засяга всеки ред от програмния интерфейс. Дървото от изгледи съдържа обратни връзки родител ↔ дете и абонаменти за събития, а средата на еталона ги разрешава чрез автоматично събиране на боклука, каквото тук няма.

2.4.1. Разгледани варианти

(а) **Споделено притежаване навсякъде.** Най-близко до поведението на еталона и най-лесно за писане. Отхвърлено, защото цикълът „контрола притежава обработчик, който улавя контролата“ остава напълно безшумен: паметта никога не се освобождава, а инструментите за откриване на течове мълчат, защото обектите са *достижими* — просто взаимно.

(б) **Арена с индекси вместо указатели.** Решава цикъла по начина на изграждане и дава компактно разположение в паметта. Отхвърлено по К2: заменя типовете с идентификатори, при което програмният интерфейс губи типова информация и грешка в идентификатор става невъзможна за откриване от компилатора.

(в) **Разпределение на ролите между четири средства.** Приетият вариант.

2.4.2. Приетият модел

Притежаващият манипулатор е **само преместваем**, наблюдаващият е копируем, а съвместното притежаване се получава само чрез изричен метод. Следствието: най-честият начин за цикъл — улавяне на притежаващ манипулатор в обработчик, който самата контрола притежава — става *грешка при компилация*, не безшумено и постоянно задържане на паметта, което инструментите за откриване на течове не засичат.

2.4.3. Емпирична обосновка

Възражението срещу само преместваем манипулатор е ергономично: допълнително действие при достъп до задържана контрола. Проверено е чрез преброяване в кодовата база, не преценено:

Наблюдение	Брой
места за свързване към събитие, улавящи обхващания обект	322 от 392
места, улавящи контрола чрез копие на притежаващ манипулатор	0
места с действителен достъп до задържана контрола, върху които пада цената	111–130

Предимството на копируемия манипулатор е предимство над случай, който на практика не се среща, докато цената му — безшумният цикъл — се среща лесно. Преброяването обхваща примерните приложения и тестовите в хранилището към момента на решението: измерва как *се пише* с интерфейса, не как *би могло*.

2.4.4. Съпоставка с наличната практика

Съпоставката подкрепя избора и от друга посока: нито един зрял *native* инструментариум без автоматично събиране на боклука не предоставя *копируем притежаващ* манипулатор към контрола — всички въвеждат притежаване от родител към дете и отделен тип за наблюдаваща препратка. Системите с копируем манипулатор са подпрени или от събирач, или от проверка на заемането. Приносът тук не е в откриването на разпределението, а в това, че е *проверимо от компилатора*: същата препоръка, но като грешка при компилация вместо бележка в документацията.

2.5. Избор на платформени слоеве и на реда им

Реализирани са пет слоя. Редът, в който са изградени, е част от решението:

1. **Слоят без екран** — първи. Няма *native* инструментариум и служи като определение за това какво изобщо трябва да реализира един платформен слой. Определящ по K1: докато не съществува, нищо от преносимата част не е автоматизирано проверимо.
2. **Настолният слой на разработващата машина** — втори, защото доказва границата между преносимото и платформеното от край до край върху истински контроли.
3. **Мобилният и настолният вариант на същия инструментариум** — трети, преизползва по-голямата част от втория.
4. **Настолният инструментариум на Windows** — четвърти.
5. **Android** — последен и най-скъп по K5: границата минава през чужд обектен модел с отделно управление на живота на препратките.

Изборът кой слой да е втори следва от K5: смесеният език дава най-прекия достъп до *native* обектите — една единица за превод вижда двата свята, без междинно представяне.

2.6. Подход към декларативното описание

Езикът не предоставя рефлексия, а еталонната реализация я използва точно за откриване на обработчици, внедряване на зависимости и създаване на обекти от описанието. Изборът тук не е стил, а принудителен по същество:

1. **Тълкуване по време на изпълнение** — изисква регистър на типовете, който така или иначе трябва да бъде попълнен явно; грешките в описанието се откриват при стартиране.
2. **Пораждане на код при компилация** — грешките се откриват при компилация, няма разбор по време на изпълнение.
3. **Отказ от описание** — само програмен интерфейс.

Избран е вариант (2) — преобразуване по време на компилация — с вариант (1) като резервен път там, където езиковото средство не е налично. Компромисът е конкретен: печели се откриване на грешките при компилация и липса на разбор при стартиране; губи се възможността описанието да се изменя без повторно компилиране — съществено при инструмент за бърза разработка, тук не е.

Явната регистрация е неизбежната цена на липсата на рефлексия: регистър на обработчиците (кой договор с кой обект се обслужва), регистрация на свойствата (кое име към кой описател) и регистър на преобразувателите (как текст става стойност от даден тип) — всяко в оригинала получено чрез обхождане на типовете при изпълнение.

Отбелязва се и капан от хода на работата: при самостоятелна регистрация (статичен обект, вписващ се в регистъра при зареждане) свързващият редактор изхвърля единица за превод без нито една препратка — и регистраторът никога не се изпълнява. Решението: тези единици се събират в обектна библиотека, която не подлежи на такова изхвърляне.

2.7. Обобщение на направения избор

Таблица 3. Обобщение на петте решения по критериите K1–K5

Решение	Избран вариант	Решаващ критерий
целеви език	C++26	K5 — пряк достъп до три от петте инструментариума
стратегия	послойно, водено от тестове	K1 — само тя оставя общо описание, тоест контролно условие
собственост	четири роли, само преместваем притежател	K4 — цикълът става грешка при компилация
платформени слоеве	пет, започвайки без екран	K1 — без слоя без екран нищо не е проверимо автоматизирано
декларативен слой	преобразуване при компилация	K1 + K3 — грешките се откриват по-рано, няма разбор при пускане

Две наблюдения върху (Табл. 3). **Най-скъпото решение** е моделът на собствеността: единственото, което засяга всеки ред от публичния интерфейс, и единственото, чиято промяна по-късно би означавала пренаписване, не преработка. **Най-определящото за останалите** е слой без екран — докато не съществува, никое от другите решения не може да бъде проверено, а решение, което не може да бъде проверено, на практика не е взето.

Забележимо е, че K1 е решаващият критерий в три от петте реда — не съвпадение: работата изследва твърдение за еквивалентност, и решение, което затруднява проверката, отнема от нея повече, отколкото дава.

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

Разделът излага проектните решения на системата — слоестата структура, договорите, табличното преобразуване, алгоритмите за разположение, системата от свойства — заедно с алтернативата, спрямо която всяко е взето: интерфейсът остава преносим, а платформено обусловената част е сведена до необходимия минимум.

3.1. Синтез на общата структура

Системата е разделена на девет слоя, подредени по посока на зависимостите: по-голям номер може да зависи от по-малък, никога обратното. Правилото не е стилово — то прави преносимата част преносима: платформена частност не може да проникне в алгоритъм, който трябва да дава един и същ резултат навсякъде.

0. **Графични примитиви** — цвят, точка, размер, правоъгълник, геометрия, платно.
1. **Договори** — интерфейсите на виртуалните изгледи и техните обработчици.
2. **Обработчици (handler-и)** — таблиците за преобразуване.
3. **Платформени слоеве** — свързването с native инструментариума.
4. **Разположение** — алгоритмите за измерване и подреждане.
5. **Контроли** — програмен интерфейс, свойства, свързване на данни.
6. **Декларативен слой** — преобразуване на описание в обектен граф.
7. **Услуги на устройството** — независими от графичния стек.
8. **Инициализация** — изграждане на приложението и регистрация.

Структурата е показана на (Фиг. 2).

Таблица 4. Съответствие директория → слой → обем

Директория	ЕП	Слой
graphics/	27	0 — примитиви
core/	64	1 + 2 — договори и обработчици
layouts/	8	4 — алгоритми за разположение
controls/	159	5 — програмен интерфейс, свойства, свързване
xaml/	35	6 — декларативен слой
essentials/	40	7 — услуги на устройството
hosting/	9	8 — инициализация
animations/	6	5 — анимации
platform/	318	3 — пет платформени слоя

ЕП = единици за превод (.cpp/.mm); публични заглавни файлове — 716.

Две наблюдения: обработчиците живеят заедно с договорите, не в отделна директория, защото се изменят синхронно; и платформените слоеве са най-обемната част — количествен израз на твърдението, че платформената обвързаност е неотстранима, не подлежаща на абстрахиране.



Фигура 2. Слоеста структура и посока на зависимостите. Само слой 3 е обвързан с native инструментариум; тази граница прави останалите слоеве преносими и проверими без екран.

3.2. Проектиране на слоя за графични примитиви

Слоят съдържа стойностните типове, с които говорят всички останали слоеве: цвят, точка, размер в дробни координати, правоъгълник, радиус на закръгление, матрица на преобразуване, път и четки (плътна, линейна и радиална градиентна, шарка). Той е напълно преносим — без платформена зависимост — затова изграждането започва от него: всеки следващ слой се нуждае от неговите типове, той — от нищо.

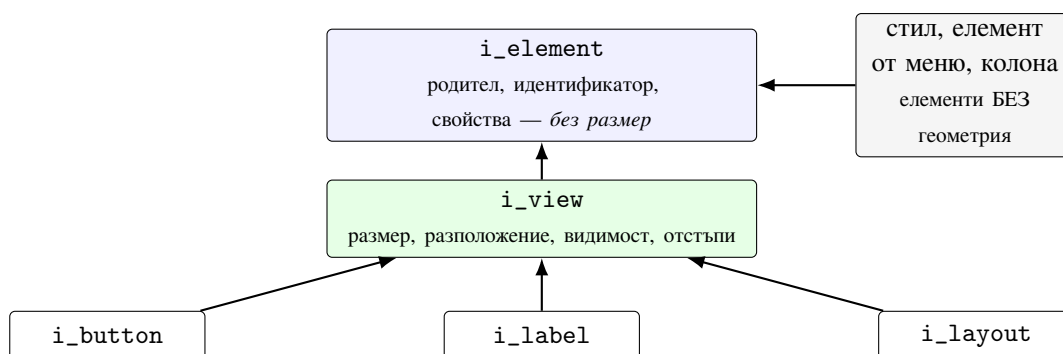
Абстракцията за рисуване (canvas) е интерфейс за команди — линия, път, запълване, текст — не за конкретна повърхност. Наред с платформените реализации съществува и *записваща*: не рисува, а запазва получените команди, което позволява да се провери *какво* е поискано, без екран.

Пример: превръщането на цвят от текстов запис в стойност е `constexpr` — изчислено при компилация, а в оригинала задължително при изпълнение — така че в артефакта попада готовата стойност, без разбор при пускане и без възможност невалиден запис да оцелее до изпълнение.

3.3. Договори на виртуалните изгледи

Виртуален изглед е обект със свойства, без рисуване. Йерархията има три нива: `i_element` (най-общият елемент — родител, идентификатор, участие в системата от свойства, но *без* размер), `i_view` (добавя геометрия: желан размер, разположение, видимост, прозрачност, отстъпи) и специализираните договори (`i_button`, `i_label`, `i_layout`, ...), които добавят само своето. Договорът описва *какво* има контролата, никога *как* изглежда — позволява един и същ договор да бъде удовлетворен от разнородни native контроли.

Разделянето на „елемент“ от „изглед“ не е излишно: не всеки елемент от дървото има размер. Дефиниция на стил, елемент от менюто, колона в решетка участват в дървото, но не се измерват. На най-общото ниво всеки от тях щеше да носи неизползвани свойства и — по-лошо — да може да получи размер.



Фигура 3. Йерархия на договорите: геометрията е на *i_view*, не на *i_element* — елементите вдясно нямат размер и не могат да получат такъв.

3.4. Проектиране на слоя за преобразуване

Обработчикът е повтарящата се единица на системата: за всяка контрола свързва виртуалния изглед с native контролата и пренася стойностите между тях.

3.4.1. Таблица „свойство → действие“

Преобразуването е *таблично*: обработчикът притежава карта от свойство към функция, прилагаща текущата му стойност върху native контролата, и втора карта за императивните операции. Смяна на стойност предизвиква съответното действие; свързване на нов изглед — пълно преминаване през таблицата.

Картите се *верижат* — на контрола, на изглед, на елемент — и разрешаването на припокриване заслужава внимание: ключ, предефиниран в по-близка карта, надделява по *съдържание*, но се изпълнява на *позицията*, на която стои в най-далечната карта, където се среща. Редът на прилагане остава стабилен — действие, което трябва да предхожда друго (например установяване на контейнера), не се измества назад само защото по-близък слой е предефинирал друго свойство. Пример: бутон, предефиниращ background, изпълнява *своята* реализация, но на позицията на background в таблицата на изгледа — преди text.

Съзнателни опростявания спрямо оригинала (без кеш на слетите таблици, без ключ за пакетно прилагане) не променят *кои* действия и в *какъв ред*, само колко работа се повтаря.

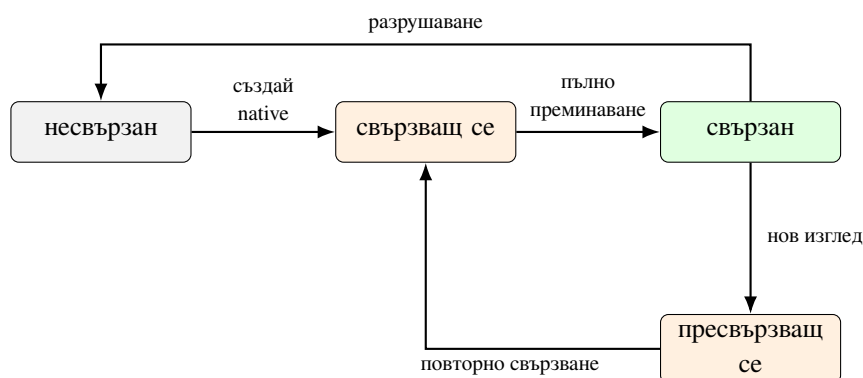
3.4.2. Обобщен базов клас без виртуални шаблони

Базата на обработчиците е шаблон, параметризиран с производния тип, типа на виртуалния изглед и на native контролата — вместо виртуален базов клас с шаблонни

методи, защото шаблонен метод не може да бъде виртуален: обобщената част се разрешава при компилация, полиморфизмът — през нешаблонен интерфейс. Незадължителните точки за разширение се откриват чрез изискване за наличие на израз, така че производният клас изброява само онова, което наистина предоставя. Интерфейсът на обработчика се наследява *виртуално*, защото обработчик на контейнер трябва едновременно да удовлетворява общия интерфейс и интерфейса за контейнери — иначе би получил два подобекта на общия интерфейс.

3.4.3. Жизнен цикъл и собственост

Жизненият цикъл има четири състояния — *несвързан*, *свързващ се*, *свързан*, *пресвързващ се* — показани на (Фиг. 4); повторно подаване на същия изглед е операция без действие. Състоянието „свързващ се“ е нарочно *видимо* за платформените действия: позволява им да пропуснат работа по свойства със стойност и без това подразбираща се, което при първото преминаване е повечето от тях — без този признак всяко свързване би изпълнило пълния набор действия, включително онези, които не променят нищо.



Фигура 4. Състояния на обработчика. „Свързващ се“ е видимо за платформените действия, за да пропуснат работа по свойства със стойност по подразбиране — при първото преминаване това е повечето от тях.

Собствеността е насочена: изгледът притежава обработчика, обработчикът — *native* контролата; обратните връзки са непритежаващи, следствие от липсата на автоматично събиране на боклука (§3.7).

Капан, установен в хода на работата: контрола с обработчик изисква вписване в *два* списъка — регистъра на типовете и списъка, изграждан при инициализация. Вписана само в единия, тя се компилира и стартира без грешка, но не се изобразява: липсващата регистрация е данна, не тип, и компилаторът не може да я улови.

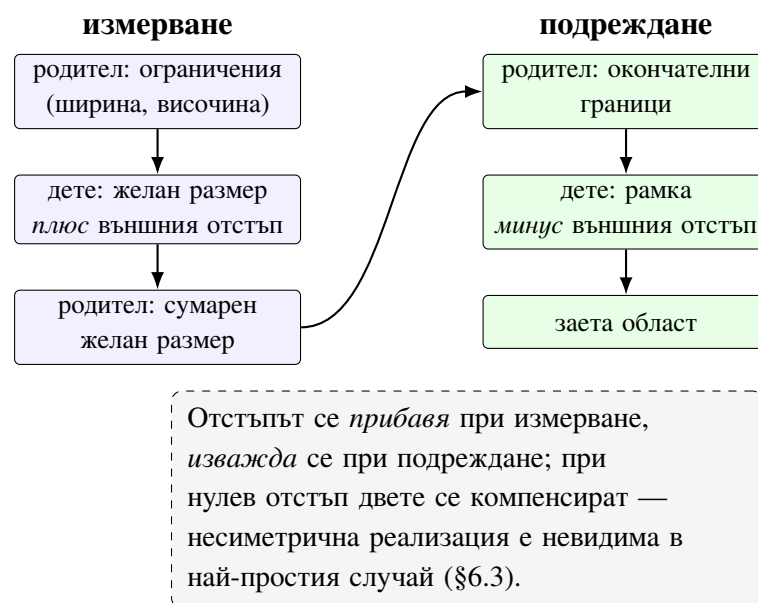
3.5. Проектиране на алгоритмите за разположение

Разположението се изчислява в преносимия слой, не се възлага на native инструментариума — определящо решение: платформената контрола се третира като правоъгълник с даден размер, не като участник в собствената си подредба. Цената е точно възпроизвеждане на алгоритмите; ползата — едно и също разположение навсякъде, проверимо без екран.

3.5.1. Двухазният модел

Интерфейсът на един алгоритъм се състои от две операции: *измерване* връща желания размер по дадени ограничения за ширина/височина, *подреждане* разполага децата по дадени граници и връща действително заетия размер. Разделянето е необходимо, защото размерът на родителя зависи от децата, а положението на децата — от окончателния размер на родителя ((Фиг. 5)).

Ограничението „неограничено“ се предава като безкрайност — „вземи толкова, колкото ти трябва“ — и се използва от подвижните области, които не ограничават децата си по посоката на превъртане: дете, което иначе разпределя оставащото място пропорционално, тук се свежда до собствения си желан размер. През вложени контейнери двете фази се разпространяват рекурсивно; дълбочината на дървото умножава цената — най-натоварената част от преносимия слой, мястото на очакваната разлика по хипотеза H2b-2.



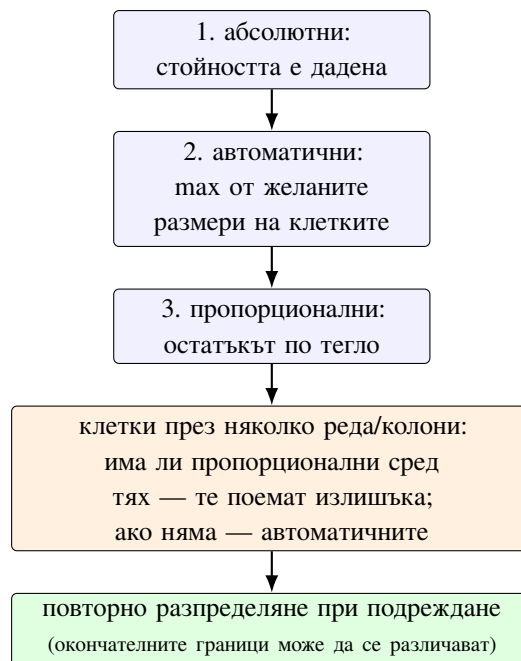
Фигура 5. Двухазният модел на разположението и симетрията на външния отстъп между двете фази.

3.5.2. Разпределяне на пропорционалните измерения

Разпределянето на измеренията в решетка е най-сложният алгоритъм в системата — на порядък по-обемн от всеки друг мениджър на разположение. Трите вида измерение се разрешават в строг ред:

1. **Абсолютно** — стойността е дадена; разрешава се веднага.
2. **Автоматично** — размерът е максимумът от желаните размери на клетките в реда/колоната; изисква измерване на децата и затова следва абсолютните.
3. **Пропорционално** — оставащото място след първите две се разпределя между тях по тегло.

Усложнението идва от клетки, разпростиращи се върху няколко реда/колони: желаният им размер се разпределя между обхванатите по различно правило — пропорционалните поемат излишъка, ако ги има сред тях, иначе го поемат автоматичните. Разпределянето се извършва *отново* при подреждане, защото окончателните граници може да се различават от измерените.



Фигура 6. Разрешаване на измеренията в решетка. Редът е задължителен: пропорционалните се разрешават едва след абсолютните и автоматичните.

3.5.3. Координатна система при подреждане

Изборът на координати при подреждане — абсолютни или относителни спрямо контейнера — изглежда като подробност и не е такава: при вложен контейнер с ненулево начало погрешният избор води до двукратно прибавяне на отместването, проявяващо се само при влагане (клас в систематизацията, §6.3).

3.6. Проектиране на системата от свойства

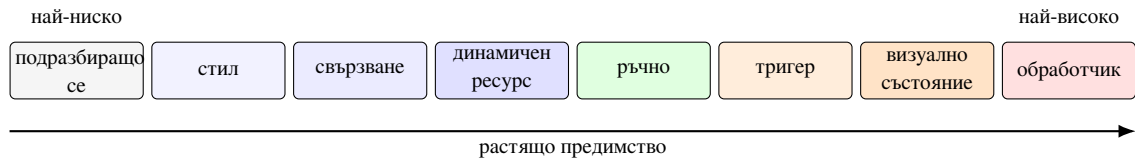
Системата от свойства съхранява стойностите, известява за промяна и разрешава предимството между едновременно валидни източници на една и съща стойност.

3.6.1. Типизиран описател вместо изтрит тип

Описателят на едно свойство (име, стойност по подразбиране, обратни извиквания) е *шаблон по типа на стойността*, за разлика от оригинала, изграден върху общия базов тип на средата за изпълнение. Следствие: стойността по подразбиране, сравнението за равенство и обратните извиквания са типизирани, без опаковане на стойностни типове и без информация за типа при изпълнение.

3.6.2. Предимство на стойностите като едно цяло число

Предимството е представено като опакована в едно 64-битово число ключова стойност, чийто *целочислен ред съвпада с реда на предимството*. Възходящо:



Фигура 7. Нива на предимство при разрешаване на стойност. Подредбата им е целочислена. Стойността от обработчика стои най-високо, но се измества от всяко изрично присвояване — отразява какво е направила native контролата, не какво е поискало приложението.

Решението има две следствия: сравнението на две предимства е едно целочислено сравнение, а стойността е изчислима при компилация, така че наименуваните константи се инициализират статично, без неопределения ред между единиците за превод.

3.6.3. „Незададено“ срещу „зададено на стойността по подразбиране“

Разграничението изглежда академично, но има пряк визуален ефект: контрола с незададен цвят на текста трябва да получи подразбиращия се на платформата, докато контрола с изрично зададен цвят, съвпадащ с подразбиращия се, трябва да получи точно него. Проверка на стойността вместо на наличието ѝ обърква двата случая — грешката, която прави текст невидим при тъмна тема (клас в §6.3).

3.7. Проектиране на потребителския програмен интерфейс

Липсата на автоматично събиране на боклука е най-същественото разминаване спрямо оригинала, защото дървото от изгледи съдържа обратни връзки родител ↔ дете и абонаменти за събития. Приетият модел разпределя работата между четири роли:

Роля	Средство	Семантика
конструиране	изграждащи функции	връщат притежаващ манипулатор върху поддървото
притежаване	<code>view_ref<T></code>	само преместваем; точно един притежател
съхраняване	<code>weak_ref<T></code>	копируем, непритежаващ; безопасен достъп
изменение	<code>observable<T></code>	реактивно; изменя се <i>данните</i> , не изгледът

Решението притежаващият манипулатор да *не* е копируем прави невъзможен най-често допусканият клас грешки: улавяне на притежаващ манипулатор в обработчик, който самата контрола притежава. Такъв цикъл не се съобщава от нито един инструмент, защото обектите са достижими — само един от друг; при копируем манипулатор това е валиден код, тук е грешка при компилация. Цената в ергономичност е реална, но ограничена до достъпа през наблюдаваща препратка с проверка за живост (измерена в §2.4).

Където съвместното притежаване е необходимо (два елемента с независим живот, които взаимно се изменят), съществува изричен метод, създаващ втори притежател — изричен и откриваем в типа: наблюдаваща препратка обявява „наблюдател“, притежаващ манипулатор — „притежател“.

3.8. Формат на декларативното описание

Входът е дървовидно описание на елементите с атрибути. Преобразуването минава през три стъпки: *разбор* (текст → дърво от възли), *изграждане* (възли → обекти, чрез регистър на типовете) и *свързване* (изразите за свързване стават връзки към данни). Първите две се извършват при компилация, третата — при изпълнение, защото зависи от данните. Регистърът на типовете и на преобразувателите (текст → стойност) е явен и замества рефлексията: всеки тип, който може да се появи в описанието, е вписан изрично.

3.9. Проектиране на платформените слоеве

Всеки платформен слой реализира едни и същи обработчици върху различен native инструментариум. Реализирани са пет: два върху UIKit (мобилният и настолният му вариант), един върху native настолния инструментариум на същата операционна система, един върху Android View системата, един върху съвременния настолен инструментариум на Windows, и един *без екран*.

Слоят без екран не е спомагателен детайл — условието, при което поведението на преносимата част (разположение, предимство, известяване) се проверява автоматизирано, без устройство и без изобразяване. Без него проверимостта би зависела от заснемане на изображения.

Двата слоя върху една и съща операционна система споделят обвързаното с езика и с общите графични услуги (цветове, шрифтове, рисуване върху платно, дати), но не и

йерархията на контролите: различни базови класове, правила за влагане, жизнени цикли на прозореца.

Слоят върху native настолния инструментариум е несравним попикселно с останалите — изгражда интерфейса от друго семейство контроли, отколкото еталонът на същата операционна система. Изискването към него е различно (пълнота на съдържанието, съгласуваност между двата входни пътя) и той не участва в количествените таблици (§5.1): число без смисъл е по-лошо, отколкото да липсва число.

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

Разделът описва как проектните решения от Раздел III са изразени в код: организацията на изходния текст, конфигурациите за компилация, реализацията на всеки слой и — най-същественото за верността на резултата — дисциплината на разработка, която не позволява поведение да бъде измислено.

4.1. Обща структура на изходния код

Изходният текст е разделен на публични заглавни файлове и реализация, а директориите следват слоевете от §3.1: графични примитиви, ядро (договори и обработчици), разположение, контроли, декларативен слой, услуги на устройството, инициализация, анимации и платформени слоеве.

Едно отклонение от очакваното: обработчиците живеят заедно с договорите в общата директория на ядрото, а не в собствена. Причината е, че двете се изменят заедно — добавянето на свойство към договор изисква ред в таблицата на съответния обработчик — и разделянето им би създавало две места за синхронна редакция, без нищо да го налага.

Съответствието директория–слой–обем е дадено в (Табл. 4) (§3.1). Имената на пространствата следват оригинала с преобразуване на стила, а всеки публичен заглавен файл носи в коментар пълното име на изходния тип — така изходният текст остава съпоставим при проверка, без търсене по подобие.

4.2. Организация на компилацията

Компилацията е организирана като независими конфигурации, всяка в собствено дърво: слой без екран, конфигурация със статичен анализ, три платформи със санитаризери — за поведение, за нишки и за неинициализирана памет — и по една платформа за всеки платформен слой.

Разделянето поражда практически проблем: пълната проверка е N независими компилации. Организацията му затова е част от реализацията, не околност — конфигурацията без екран се изпълнява първа, за да напълни кеша на компилатора, който статичният анализ след това преизползва и плаща само за самия анализ. Компилациите са инкрементални по подразбиране; изчистването е изричен избор, не предпазна мярка.

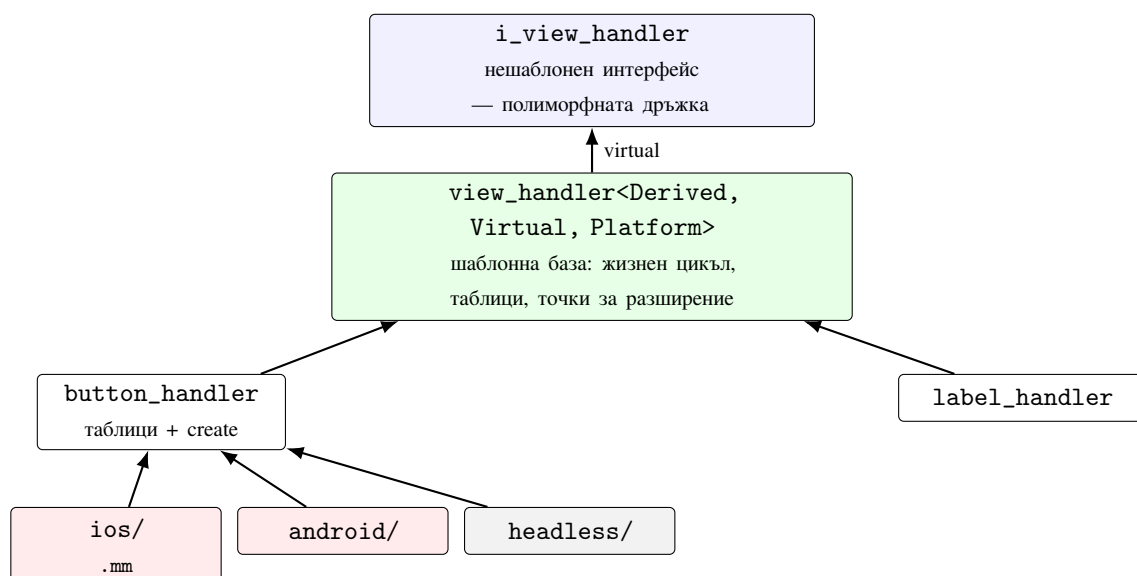
Режимите на работа са два, и смесването им е обичайната загуба на време.

Бързият вътрешен цикъл компилира една конфигурация инкрементално и изпълнява само тестовите, чиито имена съвпадат с даден израз — секунди. **Пълната проверка** минава през всички конфигурации, включително санитайзерите и статичния анализ — десетки минути.

Инструментите са нарочно два отделни, а не един с ключ, защото изборът влияе върху навика: инструмент по подразбиране бърз се използва след всяка промяна, инструмент по подразбиране пълен се използва рядко. Пълната проверка се пази за преди предаване.

4.3. Реализация на слоя за преобразуване

4.3.1. Йерархия на класовете



Фигура 8. Йерархия на обработчиците. Наследяването на интерфейса е *виртуално*, за да може обработчик на контейнер да удовлетвори едновременно него и интерфейса за контейнери без два подобекта. Платформените реализации са отделни единици за превод, а не класове — добавянето на платформа е добавяне на файлове.

Разделянето на частичен клас с платформени файлове от оригинала е изразено тук чрез отделни единици за превод: обща реализация плюс по една реализация на платформен слой, избирана при конфигуриране. Добавянето на платформа така е *добавяне* на файлове, а не изменение на съществуващи — което е и проверка дали границата между преносимото и платформеното е на правилното място.

4.3.2. Коментар на изходния текст: един представителен срез

Срезът за бутон е представителен — всяка друга контрола е със същата форма — затова се разглежда подробно веднъж:

```
1 // Текст и изглед на текста, ключирани по i_text_button.
2 property_mapper<i_text_button, button_handler>& button_handler::text_mapper()
3 {
4     static property_mapper<i_text_button, button_handler> table{
5         {"text", &button_handler::map_text},
6         {"text_color", &button_handler::map_text_color},
7         {"font", &button_handler::map_font},
8         {"character_spacing", &button_handler::map_character_spacing},
9     };
10    return table;
11 }
```

Листинг 1: Таблицата за свойства на обработчика за бутон

Четири детайла тук носят проектните решения от Раздел III. Таблицата е `static` — изгражда се веднъж за процеса, не за всеки бутон. Ключовете са низове, защото са същите имена, които системата от свойства известява при промяна. Стойностите са указатели към функции-членове, не към виртуални методи — разрешаването е по време на компилация. И типът е шаблон по *договора* (`i_text_button`), не по контролата, така че една таблица обслужва всеки тип, удовлетворяващ договора.

Платформената страна допълва картината:

```
1 std::unique_ptr<button_platform> button_handler::create_platform_view()
2 {
3     auto platform = std::make_unique<button_platform>();
4     UIButton* const button = [UIButton buttonWithType:UIButtonTypeSystem];
5     set_control_properties_from_proxy(button);
6     platform->native = (__bridge_retained void*)button; // слотът поема една
↪ препратка
7     return platform;
8 }
```

Листинг 2: Създаване на native контролата (iOS)

Тук се вижда границата между двата модела на живот: `__bridge_retained` прехвърля една препратка от управлението на средата към C++ слота, а разрушителят я връща обратно. Смесеният език позволява това да се случи в *една* функция — без междинно представяне и без слой за маршалиране.

4.4. Реализация на алгоритмите за разположение

Разпределянето на пропорционалните измерения е с порядък по-обемно от всеки друг мениджър на разположение — достатъчен довод алгоритмите да се пренасят водени от тестове, а не по описание: толкова разклонен алгоритъм има твърде много гранични случаи, за да бъдат отгнати.

```
1 double star_count = 0.0;
2 for (const definition& current : definitions)
3 {
4     if (current.is_star())
5     {
6         star_count += current.length().value();
7     }
8 }
9 return star_count;
```

Листинг 3: Сумиране на теглата на пропорционалните измерения

Функцията изглежда тривиална и точно затова е показателна: сумата се смята върху *всички* определения, не само незаетите, защото клетка, разпростряна върху няколко колони, вече е внесла своя дял. Остатъкът после се разпределя като остатък / `star_count` на единица тегло. Грешка тук — например изключване на колона с вече разрешена ширина — не се проявява при една пропорционална колона и се проявява при две с различни тегла, което е и причината този алгоритъм да се пренася воден от тестове.

4.5. Реализация на системата от свойства

Стойностите се съхраняват не като една стойност, а като *подредена по предимство* редица: всеки източник (подразбиращ се, стил, свързване, ръчно присвояване, тригер, визуално състояние, обработчик) заема свое ниво, а действащата стойност е тази с най-високото заето ниво. Изчистването на един източник не я изтрива, а разкрива следващата отдолу — точното поведение на оригинала.

Стойността, зададена от обработчика, стои на най-високото ниво, но се измества от всяко друго присвояване — защото не е избор на приложението, а отражение на това, което native контролата е направила сама, и всяко изрично присвояване трябва да я надделее.

Разрешаването на пътя при свързване е *без рефлексия*: всяко свързваемо свойство се вписва в регистър при регистрацията си, а изразът за свързване се разрешава срещу

този регистър.

4.6. Реализация на платформените слоеве

Петте платформени слоя се различават не само по инструментариум, но и по езиковата граница, през която минават.

4.6.1. Слоевете върху Apple (macOS и iOS)

Слоевете върху Apple се пишат на смесен език (.mm): една единица за превод вижда едновременно C++ и native обектния модел, а native обектът се държи пряко в C++ клас, вместо да се пренася през граница. Управлението на живота се съгласува изрично при преминаване на слота и обратно при разрушаване (листинг 11).

Защо това не превръща целия проект в смесен език. Естественото притеснение е, че щом част от файловете се компилират като Objective-C++, това ще плъзне нагоре: заглавните файлове ще носят native типове и всеки консуматор ще трябва да се компилира по същия начин. Не се случва, защото *native типът никъде не се появява в заглавен файл* — платформената структура държи native обекта зад непрозрачен указател:

```
1 struct button_platform : view_platform_base
2 {
3     void* native = nullptr;    // UIButton* / NSButton* --- типът е известен само на .mm
4 };
```

Листинг 4: Native обектът зад непрозрачен слот

Заглавният файл вижда void* и нищо повече; само реализацията — вече .mm — знае действителния тип. Следствията са три: кросплатформените заглавни файлове се компилират от обикновен C++ компилатор; тестовите без екран не се нуждаят от Apple инструментариум; и смяна на native типа не предизвиква повторна компилация извън собствената му единица за превод.

Как изграждането отделя платформената част. Разделянето е чрез *подбор на файлове*, не условна компилация вътре във файловете. Изграждането има една променлива — избраният платформен слой — и според нея включва точно една директория:

Стойност	Директория	Език на единиците
headless	platform/headless/	обикновен C++
apple	platform/apple/	Objective-C++ (UIKit)
ios	platform/ios/	Objective-C++ (UIKit)
maccatalyst	platform/ios/	същите файлове, друга целева платформа
windows	platform/windows/	C++ с проекция на компонентния модел
android	platform/android/	C++ през интерфейса за извикване

Два реда заслужават внимание. `maccatalyst` не е отделен слой — изграждането пренасочва към *същите* файлове на UIKit, само сменя целевата платформа; това е причината настолната Apple платформа да получи два различни слоя без двойно писане (§2.5). А `headless` е обикновен C++ и не изисква платформен инструментариум — затова проверката на преносимата логика се изпълнява навсякъде, дори където Apple компилатор няма.

Онова, което наистина пази разделението, е правилото от §3.1: зависимостите сочат само надолу. Ако кросплатформен заглавен файл можеше да достигне до платформен тип, нито подборът на файлове, нито непрозрачният указател биха помогнали — разделението щеше да бъде обходено още при включването.

4.6.2. Слойт върху Android

Слойт върху Android минава през интерфейса за извикване към управляваната платформа. Той е най-скъпият по K5 от §2.1 по три причини: всяко обръщение изисква търсене на метод по подпис; препратките към обекти имат два вида живот — локален (валиден до връщане от функцията) и глобален (изрично освобождаван) — и смесването им е източник на късно проявяващи се дефекти; и всяка нишка, обръщаща към платформата, трябва изрично да се присъедини и отдели.

Жизненият цикъл на дейността се съгласува с този на приложението чрез изрично препредаване на събитията — платформата може да разруши и пресъздаде дейността, без приложението да е приключило.

4.6.3. Слойт върху Windows

Слойт върху Windows минава през проекция на компонентния модел в заглавни файлове. Основното ограничение на инструментариума е върху *свободното разположение*: контейнерът с произволни координати се държи различно от този, който

поддържа сам, и системата трябва да избере правилния според описанието.

Отбелязват се три езикови капана от хода на работата, защото всеки струва часове и нито един не е документиран: имена от системните заглавни файлове, съвпадащи с имена от системата (решава се с преименуване или премахване на макрос); съкратено име на пространство в конфликт с име на тип; и невъзможността да се преопредели заделянето на памет за проектираните типове.

Обхватът на реализираните контроли тук е *частичен по замисъл* към момента на предаване: реализирани са прозорецът, страницата, контейнерите за разположение, надписът и бутонът; останалите все още огледалото без екран не изобразява нищо. Отбелязва се изрично, защото иначе би се подразбрало от таблиците — интерфейс, изграден само от нереализирани контроли, е празен и в двете колони, което е незавършена реализация, а не съвпадение.

4.6.4. Слой без екран

Слой без екран реализира същите обработчици, но „native контролата“ е обикновена структура, която само записва получените стойности. Това го прави пълноценен обект на автоматизирана проверка: всичко, което не е рисуване — разположение, предимство на стойностите, известяване, жизнен цикъл — е проверимо върху него, без устройство и без сравняване на изображения.

Той е и *определението* за това какво трябва да реализира един платформен слой: новият слой се сверява срещу него, не срещу вече съществуващ, който носи собствените си особености.

4.7. Реализация на декларативния слой

4.7.1. Входът: едно описание, два потребителя

Описанието е обикновен XML документ. Съществено е, че *същите байтове* се четат от еталонното приложение и от системата — това е условието, което прави сравнението в Раздел V валидно.

```
1 <ContentPage xmlns="http://schemas.microsoft.com/dotnet/2021/maui"
2   xmlns:x="http://schemas.microsoft.com/winfx/2009/xaml"
3   x:Class="MauiReference.Pages.BoxViewPage" Title="BoxView">
4   <ScrollView>
5     <VerticalStackLayout Spacing="6" Padding="12">
6       <Label Text="Default" />
7       <BoxView BackgroundColor="CornflowerBlue">
```



```

8         WidthRequest="160" HeightRequest="160" />
9     </VerticalStackLayout>
10 </ScrollView>
11 </ContentPage>

```

Листинг 5: Откъс от описание на интерфейс

4.7.2. Конвейерът

Описанието влиза в единицата за превод чрез `#embed` — директива на C++26, която вгражда съдържанието на файл като масив от байтове по време на компилация. Оттам нататък всичко е обикновена `constexpr` обработка:

```

1 namespace
2 {
3     // Суровият markup, вграден от компилатора (пътят е спрямо този файл).
4     constexpr unsigned char data_binding_xaml_bytes[] = {
5         #embed "data_binding.xaml"
6     };
7     constexpr maui::fixed_string data_binding_xaml{data_binding_xaml_bytes};
8 }
9
10 std::unique_ptr<maui::controls::content_page> data_binding_page(
11     std::unique_ptr<ViewModels::greeting_view_model> vm)
12 {
13     auto page = maui::build_page<ViewModels::greeting_view_model, data_binding_xaml>();
14     page->bind_to(std::move(vm)); // BindingContext -> {Binding} изразите оживяват
15     return page;
16 }

```

Листинг 6: Вграждане и изграждане на страница по време на компилация

Ключът е сигнатурата `build_page<ViewModel, fixed_string>()`: описанието е *шаблонен параметър*, не аргумент. Компиляторът разбира документа и изгражда съответните извиквания на конструктори — без разбор при стартиране и без нито един ред кодогенерация извън езика: няма отделна стъпка в изграждането и няма породени файлове в дървото.

Трите стъпки от §3.8 се разпределят така:

Стъпка	Кога	Какво изисква
разбор	при компилация	<code>constexpr</code> обработка на вградените байтове
изграждане на графа	при компилация	регистър на типовете (замества рефлексията)
свързване на данни	при изпълнение	зависи от данните, не от описанието

Печалбата е в момента, в който грешката се открива: непознат елемент или сгрешено име на свойство спира *компилацията*, вместо да предизвика отказ при

стартване.

4.7.3. Границата на средството

`#embed` не е налична във всички компилатори, които проектът поддържа. Където липсва, същият вход минава през отделна стъпка в изграждането, превръщаща го в масив от байтове — еднакъв резултат, една допълнителна зависимост за тази платформа. Цената на целия декларативен път е измерена в §6.4 и се оказва *код*, не данни.

4.8. Обработка на събития от интерфейса

Дотук стойностите текат в една посока — от приложението към *native* контролата. Събитията текат в обратната и изискват собствен механизъм, защото *native* страната е тази, която ги поражда.

4.8.1. Механизмът от страната на C++

Няма делегати и няма събиране на боклука, затова събитието е самостоятелен тип: `event<Args...>` с `connect` (връща *жетон*), `disconnect` и `raise`. Жетонът е съществен — позволява абонаментът да бъде прекратен *детерминистично*, не при следващо събиране. Има и обвивка, която прекратява абонамента в разрушителя си, така че животът на връзката съвпада с обхвата, в който е създадена.

4.8.2. Пътят от *native* контролата до приложението

Между двата обектни модела стои малък посредник. На Apple платформите това е обект от *native* типовата система с единствена задача — да препрати:

```
1 // Obj-C trampoline: препраща target-action на UIButton към виртуалния изглед.
2 @interface MauiButtonEventProxy : NSObject
3 @property(n nonatomic) maui::core::button_handler* handler;
4 - (void)onTouchUpInside:(id)sender;
5 @end
```

Листинг 7: Посредник, който препраща *native* действието към виртуалния изглед

Веригата е: *native* действие → посредник → обработчик → `send_clicked()` на виртуалния изглед → контролата издига собственото си събитие → обработчиците на приложението.

Два детайла не са произволни. Първо, *редът* на събитията се пренася точно: при отпускане на пръста върху бутона първо се издига „отпуснат“, едва след това „натиснат“ —

редът на еталона. Второ, посредникът държи *непритежаваща* препратка към обработчика; притежаваща би затворила цикъл (обработчикът притежава *native* контролата, която държи посредника) — точно класът цикли, който моделът на собственост от §2.4 прави невъзможен по-нагоре в стека.

4.8.3. Именувани събития за декларативния слой

Описанието може да свърже действие по *име* (например атрибут Clicked). Тъй като рефлексия няма, всяка контрола вписва събитията си в регистър при изграждането си — съответствие „име → функция, която абонира“. Декларативният слой търси там, намира функцията и я извиква — същата размяна както при свойствата: явен запис вместо откриване по време на изпълнение.

4.9. Дисциплина на разработка и осигуряване на верността

Този параграф описва процеса, а не кода — поставен в раздела за реализация съзнателно, защото при работа с външен еталон именно процесът отличава пренасяне на поведение от повторното му измисляне.

4.9.1. Цикълът „по една компонента“

За всяка компонента редът е фиксиран: прочитане на договора и на *тестовете* на оригинала; пренасяне на тестовете *преди* реализацията, така че да се компилират и да се провалят; реализация до преминаването им; и чак тогава преминаване към следващата. По-важно от самия ред е правилото, което той налага, формулирано отрицателно: *когато поведението не е известно, то се прочита от оригинала — не се предполага*. Правдоподобно предположение, разминаващо се с действителното поведение, е дефект, не приблизително решение.

Трите нива на истината и въпросите, на които отговарят, са изложени в §2.3: описанието на публичната повърхност дава *формата*, тестовете — *поведението*, изходният текст — *реализацията*.

Разминаването между тях е реално и е било разрешавано. Пример: изходният текст на еталона поставя долна граница на размера на контрола; разпространената версия изобразява по-малък размер без никаква граница. Формата не казва нищо по въпроса, тестовете също. Приложеното правило — предимство има изобразеното — е записано

предварително (§5.3) и промяната е отбелязана в кода като документирано отклонение с цитиране на двата източника. Без предварително записано правило решението би било произволно.

4.9.2. Статичен анализ и санитайзери като част от цикъла

Статичният анализ се изпълнява като отделна конфигурация с изискване за *нула* находки; потискането на находка вместо отстраняването ѝ не е допустимо. Санитайзерите — за неопределено поведение, за състезания между нишки и за неинициализирана памет — са три отделни конфигурации, защото не могат да се съчетаят в една.

Всичко това е част от цикъла, не заключителна проверка: находка, открита седмици след внасянето си, изисква възстановяване на контекст, който вече не е в главата на никого, докато същата находка в цикъла, в който е внесена, се отстранява веднага. Разликата е в порядъци, не в проценти.

4.9.3. Обхват на автоматизираната проверка

Таблица 5. Обем на автоматизираната проверка по слой

Слой	Файлове	Какво покриват
графични примитиви	13	стойностни типове, геометрия
ядро (договори, таблици)	17	свойства, предимство, жизнен цикъл
разположение	9	измерване и подреждане на всеки мениджър
контроли	190	поведението на всяка контрола
декларативен слой	9	разбор, изграждане, свързване
услуги на устройството	34	—
инициализация	4	регистрация, изграждане на приложението
анимации	5	—
платформени слоеве	19	връзките към native страната

По запис от дневника на разработката към последната пълна проверка конфигурацията без екран изпълнява **3765** случая, всички преминаващи. Числото расте с всяка добавена компонента и се чете наново непосредствено преди предаване.

Съществено е *какво този обем не доказва*. Той измерва преносимата част: разположение, свойства, известяване, жизнен цикъл. Изобразяването не се проверява от него — проверява се от методиката в Раздел V. Двете мерки се допълват и нито една не замества другата: система с изцяло преминаващи тестове може да изобразява погрешно, а

система с точно съвпадение на изображенията може да има грешна семантика, която тези интерфейси просто не докосват.

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА НА ЕКВИВАЛЕНТНОСТТА

Разделът превръща твърдението за независимост от платформения инструментариум в измерима величина, от постановката на експеримента до мерките срещу недостоверно измерване — последните застъпени съзнателно най-обширно, защото основната заплаха тук не е неточност, а правдоподобна неистина.

По вид подходът принадлежи към визуалното тестване на графичен интерфейс [13], но с различна цел: там се сравняват две състояния на една система във времето (регресия), тук — две независими реализации на едно описание в един и същ момент.

5.1. Постановка на експеримента

Измерваната величина е степента на съвпадение между изображението на еталонната реализация и изображението на разработената, при идентично входно описание, устройство, тема и момент.

Хипотезата: за интерфейс, описан изцяло в общото описание, съвпадението може да се доведе до праг, отвъд който остатъчната разлика се обяснява само с изброими причини — отстраним дефект или изрично изключение (§5.3). Отхвърлянето ѝ би означавало неотстраним и необясним клас различия: граница на независимостта извън обхвата на абстракцията.

5.1.1. Едно описание, две независими реализации

Валидността на сравнението почива на *един и същ файл*, не два еквивалентни: едни и същи байтове захранват еталонното приложение и се вграждат при компилация в разработената система — разликата между изображенията не идва от входа, единствената променлива е реализацията.

Измерването обхваща 172 интерфейса, подбрани да покрият всяка контрола и всеки алгоритъм за разположение, не да представят типично приложение.

Третата колона не е излишна: разработената система има два входни пътя — програмен интерфейс и общо описание; невидяно разминаване между тях е дефект точно колкото разминаването с еталона, затова тя изгражда същите интерфейси по другия път и участва самостоятелно в оценяването.

5.1.2. Обхват на измерването и различният статут на слоевете

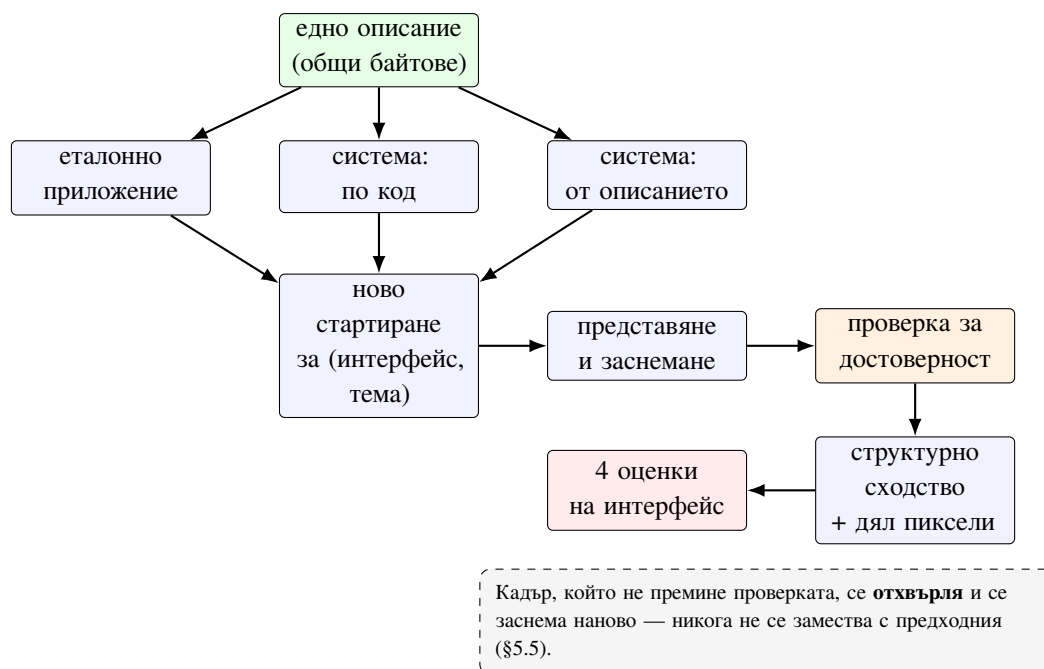
Измерването обхваща четири платформи, всяка с 172 интерфейса, две теми и две колони — четири оценки на интерфейс (§5.3). Петата платформа, с различния native инструментариум, участва само качествено: нейният слой изгражда интерфейса от друго семейство контроли, отколкото еталонът на същата операционна система, затова попикселното съвпадение там е принципно недостижимо. Изискването към него е различно — пълнота на съдържанието и съгласуваност между входните пътища, без количествена таблица; оценка там, където няма смисъл, е по-лошо от липса на оценка.

5.2. Автоматизирано заснемане

Конвейерът е показан на (Фиг. 9): виртуална машина, управлявана през отдалечена обвивка, чийто гост-агент разгръща, стартира, задвижва, представя прозореца в точен размер и заснема; оценяването остава на управляващата машина, върху изтеглените кадри.

Възпроизводимостта изисква средата да е записана: настолните платформи се снемат във виртуална машина с изрично зададена разделителна способност 1512×950 при мащаб 1:1, защото списъкът с режими съдържа и режими с двоен мащаб, при които кадърът излиза двойно уголемен и отпада от проверката за размер, без това да личи от съобщенията — задаването на режима се отчита като успешно. Мобилните платформи се снемат на симулатор и емулатор с фиксиран модел устройство.

Средите: домакин macOS 26.5.2 (управляваща машина и симулатор), виртуална машина с macOS за настолните платформи, виртуална машина с Windows 11 за WinUI и Android емулатор с API 34, срещу еталон закрепен на версия 10.0.71 — версията, спрямо която важи „предимство има изобразеното“ (§5.3). *Ограничение:* версиите на гостовите операционни системи не са закрепени; при повторение те трябва да бъдат отчетени наново, тъй като част от разминаванията (§6.3, клас 5) зависят от конкретната версия.



Фигура 9. Гръбнакът на измерването. Едно описание захранва три приложения; всяка двойка „интерфейс – тема“ се сменя от новостартиран процес; кадър, който не премине проверката за достоверност, се отхвърля и заснемането се повтаря, вместо да бъде заместено.

5.2.1. Изоляция на всяко измерване

Всяка двойка „интерфейс – тема“ се сменя от *новостартиран процес*: интерфейсът се задава чрез променлива на средата при стартиране, не чрез навигация в работещо приложение — решение заради заснемането, със следствие за Раздел VI: всяка извадка идва от студен процес, независимо от реда на минаване.

Обратната страна е ограничение на инструмента: щом всеки интерфейс се наблюдава в отделен процес, конвейерът *не може* да наблюдава освобождаване на памет при напускането му — измерва следата, не своевременността на освобождаването ѝ.

5.2.2. Задвижване на интерфейса

Задействието има два пътя. По *абсолютни координати* — натискане в точка от госта — работи винаги, но е чувствително: контрола, изместена с няколко пиксела, превръща натискането в удар по празно място, безшумно. По *идентификатор на елемент* — вграден агент по локален мрежов канал намира елемента и предизвиква действието — устойчив е на разместване, но изисква идентификатори в описанието, а не всеки интерфейс ги има.

Изборът е автоматичен: с идентификатор и отговор от агента се ползва вторият

път, иначе координати; отказът се отбелязва в дневника, за да не се скрие тиха замяна на устойчив сценарий в чувствителен.

Всеки задвижван интерфейс носи собствен *сценарий* — файл в `docs/comparison/scenarios/`, изброяващ поредица от именувани стъпки („initial“ е по договор кадърът в покой), всяка с незадължително действие (натискане, превъртане, влачене, плъзгане или въвеждане на текст) и координати като дял от заснетата повърхност. От интерфейсите на борда **38** носят такъв сценарий, повечето с повече от една стъпка; останалите се заснемат само в покой.

Изчакването преди заснемане след действие не е адаптивно измерване на покой, а фиксирана пауза, заменима поотделно за всяка стъпка. Стойността се калибрира ръчно, чрез повторение: четири последователни стартирания на еталона върху `box_view/iOS` показват при пауза 0,6 s разсейване от 52 px между собствените кацания на еталона, а при 2,0 s — и осемте стартирания на двете страни кацат в един и същ ред пиксели (коментар в `scenarios/box_view.toml`). Паузата защитава срещу същия смущаващ фактор като „кадър по време на преход“ (§5.5) — разликата е, че тук се предотвратява, а не се открива *post factum*.

5.3. Правила за оценяване

Сравнението на две изображения дава число; превръщането му в „дефект“ или „допустимо разминаване“ е решение — вземано наново всеки път, то би позволило всяко неудобно разминаване да се обясни *post factum*. Затова методиката включва *предварително записан набор от правила*, всяко формулирано веднъж, датирано и приложено механично; наборът расте само в една посока — ново наблюдение се записва като въпрос и се решава *преди* да бъде приложено, не след резултата.

5.3.1. Основното правило

Еталонът е меродавен за цялото съдържание — всяка разлика в цвят, размер, разстояние или текст е дефект на разработената система, никога „несъвършенство на еталона“; без това правило всяко разминаване би подлежало на предоговаряне и мярката би загубила смисъл.

5.3.2. Изрично изброените изключения

Изключенията от правилото са именно изброени, не изведени по преценка:

1. **Рамка на заснемане.** Еталонното приложение поставя всеки интерфейс във вътрешна рамка с голям външен отстъп и подрязва горния и долния му край — равномерно отместване на приложението-носител, не на изобразяването, което не изменя размер, цвят или вътрешен отстъп на нито една контрола; само отместването отпада от сметката, разлика в самите контроли се брои.
2. **Особеност на конкретен native инструментариум, която еталонът също търпи.** На една от настолните среди еталонът не изобразява точките на индикатор, определени вградени изображения и сянка, докато на другите две ги изобразява; системата ги изобразява навсякъде — възпроизвеждането на липсата би било грешка, системата е по-вярна на описанието.
3. **Артефакт на измерването, а не на изобразяването (§5.5).**

Всяко изключение е *ограничено* до конкретна контрола и среда, не общо разрешение за разминаване — изключение, което важи навсякъде, е отказ от мярката.

5.3.3. Противоречие между документацията на еталона и поведението му

Отделно правило урежда случая, в който изходният текст на еталона казва едно, а действително изобразеното — друго, защото изходният текст е снимка от момент, а измерването е срещу разпространената версия. Правилото: **предимство има изобразеното** — изходният текст остава източник за *механизма*, не за стойност, променена оттогава; разминаването се отбелязва в кода с посочване на двата източника.

5.3.4. Четирите задължителни сравнения

Еталонът се съпоставя *независимо* с всяка от двете колони на разработената система, в двете теми — четири оценки на интерфейс:

№	Еталон	Съпоставена колона	Тема
1	светла	изграждане по програмен интерфейс	светла
2	светла	изграждане от описанието	светла
3	тъмна	изграждане по програмен интерфейс	тъмна
4	тъмна	изграждане от описанието	тъмна

Четири оценки **не се осредняват**: това би скрило точно интересния случай — единият входен път верен, другият не; средното на „вярно“ и „невярно“ изглежда „почти вярно“, погрешно описание на положението.

5.4. Количествено оценяване

Оценяването е детерминистично: две допълващи се мерки, изчислени за всяка двойка изображения.

Структурно сходство [14] — чувствително към промени в структурата, слабо чувствително към равномерни отмествания на яркост; подходящо, защото интересът е към разлики във *формата и разположението*.

За два съответни прозореца x и y мярката е

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}, \quad (1)$$

където μ е средната яркост, σ^2 — дисперсията, σ_{xy} — ковариацията между прозорците, а $C_1 = (K_1L)^2$, $C_2 = (K_2L)^2$ са стабилизиращи константи при $L = 255$, $K_1 = 0,01$, $K_2 = 0,03$; трите множителя отговарят на *яркост*, *контраст* и *структура*, а оценката за изображението е средното на $\text{SSIM}(x, y)$ по всички прозорци, върху яркостния канал. Равномерно отместване на яркостта изменя μ , но оставя σ_{xy} и σ непроменени, така че структурният множител не се влияе — изместване на контрола обаче руши ковариацията и мярката пада рязко.

Дял на видимо различаващите се пиксели. Втората мярка е

$$D = \frac{1}{N} \sum_{i=1}^N \left[\max_{c \in \{R, G, B\}} |a_i^c - b_i^c| > \tau \right], \quad (2)$$

където N е броят пиксели, a_i, b_i — съответните пиксели, $[\cdot]$ е индикатор, а τ е прагът, отделящ разлика от шум при заглаждане на шрифта; максимумът по канали, не средното, е съзнателен — разлика само в синьото например е разлика, а средното я разрежда до незабележимост.

Двете се отчитат заедно: сходството остава високо при дребна, систематична разлика в цвят, която делът на пикселите улавя; делът остава нисък при отместен, но значим елемент, което сходството наказва — всяка мярка поотделно има сяло петно, заедно нямат общо.

Праговете, приложени към *по-лошата* от двете теми, не средната:

Оценка	Структурно сходство	Различаващи се пиксели
съвпада	$\geq 0,98$	и $\leq 1,0 \%$
незначителна разлика	$\geq 0,90$	и $\leq 8,0 \%$
съществена разлика	всичко останало	

Двете условия важат *едновременно*. Пикселът се брои различаващ се при разлика по някой канал над **25** от 255 — праг, поглътцащ и шума от заглаждане на шрифта, тоест наказващ еднакво изобразен текст. Калибрирани са срещу две наблюдения: долната граница за „съвпада“ държи заглаждането на шрифта под себе си при иначе идентично разположение; горната за „незначителна“ пуска изместена или липсваща контрола над себе си, като съществено разминаване.

Втора проверка — по колко интерфейса всяка мярка поотделно би дала различна оценка — идва от `comparison.json`: от **688** двойки, при **8** самостоятелно различна е оценката на пикселния дял, при **3** — на сходството, никога и двете едновременно. Малките, ненулеви числа в *двете посоки* показват, че премахването на която и да е мярка би сменило оценката на единадесет интерфейса — при разминаване само в едната посока по-простата мярка щеше да стигне.

5.4.1. Оценяване на подвижни интерфейси

Задвижена или анимирана страница не дава една двойка изображения, а две *последователности* от кадри — по една на колона — и мерките по-горе не казват какво да се прави с втора, трета, четвърта двойка. Оценяването минава през пет стъпки:

1. **Сдвояване по стъпка.** Кадрите от двете колони се сдвояват по името на стъпката, която ги е произвела; кадър без чифт *отпада*, вместо да се сдвои с най-близкия по ред — отпадналите се преброяват и се отчитат изрично, вместо мълчаливо да намалят извадката.
2. **Времево подравняване.** Едно общо отместване (до 3 кадъра) компенсира разлика в скоростта на заснемане между двете приложения; отхвърля се, ако би задържало под 60 % от сдвоените кадри — страница, обходена в грешен ред, не може да бъде подравнена към съгласие.
3. **Пространствено подравняване.** Отделно, ограничено до 48 px по вертикала, компенсира инерционно разминаване в кацането на превъртане — прилага се *само*

ако кадърът в покой преди действието вече съвпада с борда, и никога не връща по-лоша оценка от неподравнената; изместване извън тази граница остава непоправено и се отчита като разлика.

4. **Собствено движение.** Всяка колона се сравнява *сама със себе си* — първият ѝ кадър срещу най-различния измежду останалите — за да раздели „и двете страни стоят неподвижни“ от „еталонът анимира, системата замръзва“: кръстосаното структурно сходство не забелязва малък въртящ се индикатор върху иначе еднакъв кадър.
5. **Устойчив край, при неуспех на кръстосаното сравнение.** На платформата, на която движението *не е* възпроизводимо между стартирания¹, вдвояването по стъпка е твърде шумно, за да реши само. Преди да съществуваше тази стъпка, такъв случай оставаше с горна граница жълто, никога зелено. Оценката пита вместо това дали *последните три кадъра* на всяка колона са спрели да се менят помежду си — и само ако и двете страни са спрели, сравнява установеното им състояние едно с друго, с прощаването от точка 3. Страница, която никога не спира (например пулсиращ индикатор), остава без принудена оценка, вместо погрешно зелена или червена.

Точка 5 заменя единствено *оценката на движението* в запис на клетката — не цвета ѝ: класификацията на цвета никога не се повишава само от заключение за движение. Разграничението пази основното правило на §5.3 — еталонът остава меродавен — дори когато механизмът, потвърждаващ *защо* дадено разминаване не е дефект, е по-сложен от статично сравнение на два кадъра.

Класификацията разчита и на *еталонът срещу себе си*: едно и също, непроменено приложение на еталона, стартирано неколkokратно и задвижено по един и същ начин; собственото му разсейване между стартирания служи за шумов праг — разлика, не по-голяма от него, не може да бъде приписана на портираната система. Сравненията се групират по един и същ ангажимент от двете страни: по-ранен опит без това групиране обърка прекомпилиране на системата между заснемания с истинска нестабилност и докладва грешна цифра — същият клас смуцаващ фактор, който §5.5 каталогизира за статичните кадри.

¹Android — измерено разсейване между собствени стартирания $\approx 0,65\%$, срещу $\approx 53,3\%$ разлика между кръстосано сдвоени кадри на един и същ момент от жеста: 80 пъти по-голямо.

5.4.2. Отказ от оценяване чрез езиков модел

Първоначално оценките включваха и оценка чрез езиков модел — *изоставена*, след като две независими такива оценки на едни и същи двойки се разминаха помежду си и с измеримото. Причината е методологична, не техническа: невъзпроизводима оценка не служи за мярка, колкото и полезно описание да дава. Отзивът остава полезен при *диагностика* на защо две изображения се различават, не при *оценяване*.

5.5. Достоверност на измерването

Този параграф разглежда основната заплаха пред изследването: заснемач конвейер, който докладва „заснети 1104, откази 0“, твърди само, че файловете са записани, не че показват онова, което претендират. Описаните по-долу режими на отказ са настъпвали в хода на работата и нито един не е бил сигнализиран от самия конвейер.

5.5.1. Заснемане, което произвежда правдоподобна неистина

Кадър от чужда колона. Когато представянето на прозореца се провали, а предходният файл остане на мястото си, изтеглянето връща предишния файл под новото име — валидно изображение, което сравнява реализацията със самата себе си или с чужда тема. Установеният случай доведе до около 80% съвпадение за интерфейс без никакъв дефект, непоправимо с промяна в реализацията.

Съвпадащи байтове между теми. Ако темата не е приложена при стартиране, „тъмното“ заснемане е второ светло — признак, точен защото фонът винаги се различава между теми: два байт-идентични кадъра на един интерфейс са сигнал за дефект, не рядкост.

Кадър по време на преход. Заснемане преди установяване на изобразяването — и трите минават всяка проверка, основана само на наличие и размер на файла.

5.5.2. Механични проверки вместо доверие

Контролът е механичен и се извършва *преди* оценяването:

- хеширане на всички кадри и откриване на съвпадащи байтове между колони (винаги дефект) и между теми в една колона (дефект, освен ако интерфейсът не е обявен изрично за независим от темата);
- изискване неуспешното представяне да *отпадне* като кадър, вместо да бъде заместено

с предходния;

- изискване двете колони на едно сравнение да произхождат от едно и също изпълнение.

Изведеното от това правило е формулирано отрицателно: *при съмнение кадърът се отхвърля, а не се приема*. Всеки резервен вариант при заснемане произвежда измислица — разликата е само в това колко правдоподобна.

5.5.3. Смущаващи фактори, установени в хода на работата

Трите фактора по-долу първоначално са били диагностицирани като дефект на реализацията, преди да бъдат разпознати.

1. **Смяна на денонощието между двете заснемания.** Интерфейс с дата или час изменя ширината на изображения низ, имитирайки дефект в разположението — съпоставими са само интерфейси, чиито колони идват от едно изпълнение.
2. **Различен мащаб на екрана.** Заснемане на еталона при увеличен системен мащаб поражда цял клас „еталонът е по-едър“, който не е дефект — опит да се „поправи“ води до влошаване на реализацията.
3. **Източник на темата.** Темата произхожда от операционната система, не от настройка на приложението; без изрично управление двете колони се снемат в различни теми.

Към същия списък спада системната лента на една от платформите: час и заряд се различават между заснемания без общо с изобразяването — изключва се чрез отрязване на горната ивица преди оценяването, около 135 реда, определени чрез измерване, докато съдържанието под нея се подравнява.

5.6. Предварителна регистрация на хипотезите

Хипотезите за цената на реализацията са записани *преди* първото измерване, заедно с изискванията за валидност. Практиката е установена в емпиричната софтуерна инженерия като *регистриран доклад* [15]; тук е приложена в опростен вид — регистрацията е част от документацията на хранилището, датирана.

Изискванията за валидност, договорени предварително:

- еднаква конфигурация на компилация от двете страни — отстраняване на грешки срещу издаване е негодно сравнение в двете посоки;
- повторения и разпределения, не единични числа;

- времето за кадър по горни проценти, не по средна стойност — забавянето е явление на опашката на разпределението;
- разграничаване на студено от топло стартиране;
- определение за „стартиране“, независимо от реализацията — до първия заснет кадър, не началния екран, а не до появата на прозорец, различно предхождащ изобразяването в двете реализации;
- неизмерим слой се отчита като *неизмерен*, не се осреднява с останалите.

Регистрацията включва и хипотеза **срещу** очакването на автора — за величина, при която се очаква да *не* се установи разлика; потвърждаването ѝ е резултат, не пропуск — изследване с изцяло сбъднати прогнози се чете като застъпничество.

Пълният текст, с датите и формулировките в оригиналния им вид, е приложен към работата (Приложение Д), възпроизведен без редакция — преформулирана след факта прогноза не е прогноза.

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Разделът представя резултатите в следния ред: степен на съвпадение, представителни съпоставки, систематизация на нарушенията на независимостта, цена на реализацията и ограничения. Основното съдържание е в §6.3: броят съвпадащи интерфейси е състояние на конкретна реализация към конкретна дата, докато систематизацията на *къде* и *защо* абстракцията протича се обобщава извън артефакта.

6.1. Степен на съвпадение

Обобщението е в (Табл. 6). От 172 интерфейса на всяка от четирите платформи — iOS, Mac Catalyst, Android и Windows — броят на съществено различаващите се по входния път *по код* е 0, 0, 0 и 0: бордът е практически изчистен от съществени разлики на четирите платформи едновременно. Единственото изключение е по входния път *от описанието* на iOS — една двойка, разгледана в §6.2. Пълната картина е топлинна карта в Приложение В (Табл. 15), всяка клетка носи измереното структурно сходство, а цветът следва стойността (зелено 1,000 — жълто 0,75 — червено 0,50) — изключенията се намират с поглед.

Таблица 6. Степен на съвпадение по платформена лента и входен път

Лента	Входен път	съвпада	незначителна	съществена
iOS	по код	168	4	0
	от описанието	166	5	1
Mac Catalyst	по код	169	3	0
	от описанието	170	2	0
Android	по код	165	7	0
	от описанието	165	7	0
Windows	по код	170	2	0
	от описанието	170	2	0

6.1.1. Интерфейси с побайтово съвпадение

Част от интерфейсите се изобразяват от системата *побайтово идентично* с еталона — не близко, а един и същ файл по съдържание (Табл. 7).

Таблица 7. Кадри с побайтово съвпадение между еталона и системата

Платформа	светла тема	тъмна тема
iOS	45	88
Mac Catalyst	0	0
Android	35	0
Windows	51	52

Резултатът заслужава проверка, преди да бъде приет: побайтово съвпадение е и признакът на известен режим на отказ — кадър, попаднал под две имена (§5.5). Разграничението е направено с три независими проверки:

1. **Времена на файловете.** Трите колони носят различни времена, подредени по реда на заснемащите проходи — дни и часове една от друга; размножен кадър би носил един момент.
2. **Темите остават различни.** Във всичките 172 случая светлото и тъмното заснемане се различават; известният режим на отказ има точно обратния признак.
3. **Съгласуваност с останалите платформи.** Платформата, при която заснемането обхваща прозорец с жива системна лента и сянка, има *нула* такива случая — очакваното, защото там побайтова устойчивост е невъзможна по начина на изграждане.

Постижимо е тук, защото двете колони минават през един и същ заснемащ инструмент, при една и съща разделителна способност, с фиксиран часовник в лентата и детерминистичен кодер: еднакви пиксели дават еднакви байтове.

Числата в (Табл. 7) са *по-строг*, не по-слаб критерий спрямо структурно сходство 1,0000: пикселът се брои различаващ се едва след разлика над 25 от 255 по канал (§5.4), за да не наказва заглаждането на шрифта, а самото структурно сходство се показва с четири знака („1,0000“ означава $\geq 0,99995$, не точно единица) — затова кадър с оценка „точно съвпадение“ невинаги е и байт по байт същият файл; двете мерки отговарят на различни въпроси. Системната лента участва в оценяването навсякъде освен на едната мобилна платформа, където горните 140 реда се отрязват (§5.5); на настолната платформа тя носи *заглавието на прозореца* — различно по замисъл между еталон и система — и остава източник на дребни, немаскируеми разлики, независимо от съдържанието отдолу.

Следствие за инструмента, не за данните. Механичната проверка от §5.5 третира всяко междуколонно съвпадение като дефект — вярно, докато системата не достигаше точно съвпадение, но източник на лъжливи тревоги, чийто брой расте точно с качеството

на реализацията. Безусловно вярно остава по-тясното условие: междуклонно съвпадение *в съчетание със* съвпадение между темите. Критерий за достоверност може да остарее заради успех на измерваната система — основание той да се преразглежда, не да се приема за вечен.

6.1.2. Платформи с ограничена съпоставимост

Две платформи носят ограничения, отчитани отделно, не осреднени с останалите:

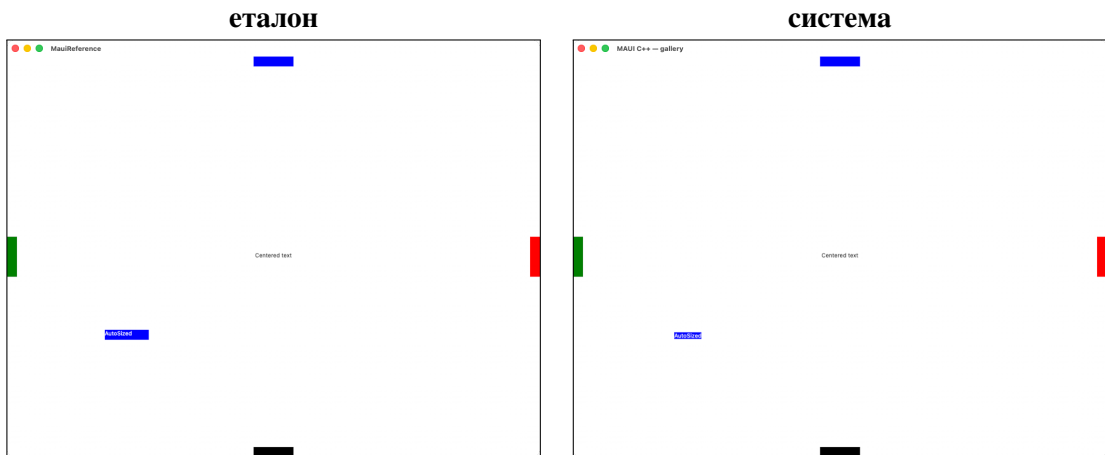
1. **Тъмната тема на една от мобилните платформи не е съпоставима.** Еталонът там изобразява светъл интерфейс независимо от поисканата тема (средна яркост на тялото 137,7 срещу 139,3 — на практика еднакви), докато системата изобразява действително тъмен; системата е права срещу неработещ еталон, но точно затова числото не носи информация.
2. **Платформата с различния *native* инструментариум** не се оценява количествено по замисъл (§5.1) — за нея се отчитат пълнота на съдържанието и съгласуваност между двата входни пътя.

6.1.3. Динамика на резултата

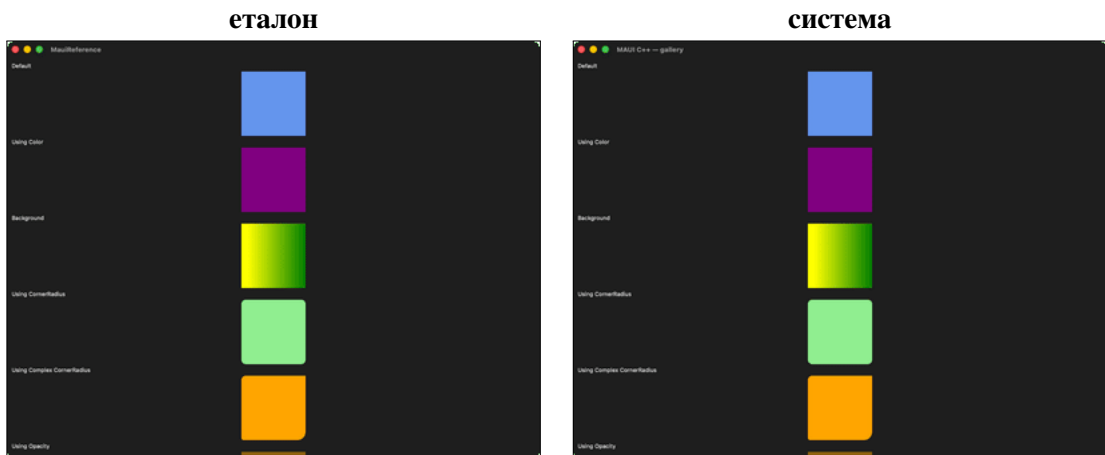
Дневникът на разработката записва състоянието на борда след всяка кампания за отстраняване на разминавания: резултатът е достигнат постепенно, а не наведнъж — само за последните девет дни от работата бяха направени 111 отделни ангажимента по заснемащата и сравняващата инфраструктура, всеки затварящ по едно установено разминаване. Записите са свободен текст, не машинно четими данни, затова тук не се привежда графика — възстановена крива от непълни записи би внушила точност, каквато източникът няма; систематизацията в §6.3 е носителят на този опит.

6.2. Представителни съпоставки

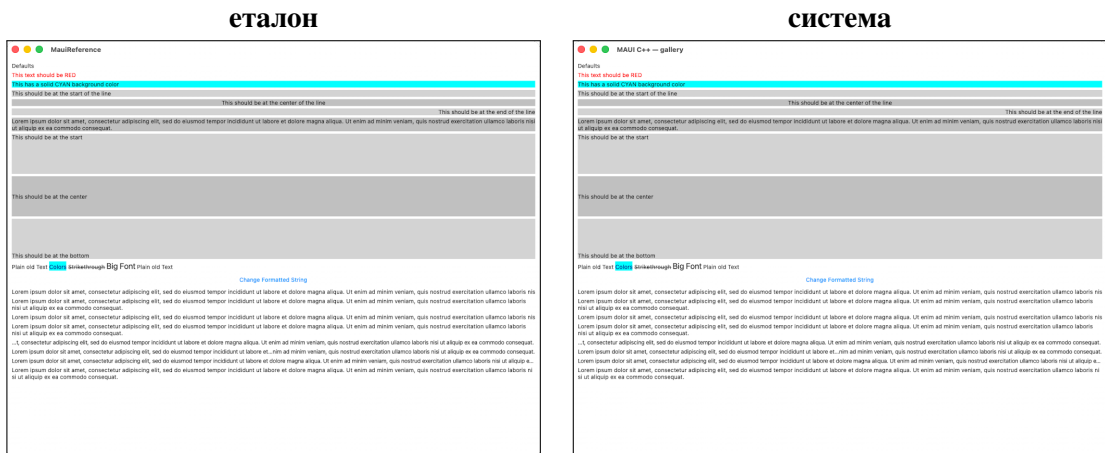
Фигурите по-долу поставят еталона и системата една до друга при идентично входно описание. Изображенията се вземат чрез символни връзки към борда (`tools/gen_tex.py`), тоест са *същите файлове*, които са били оценени — не подбрани отделно за изложението.



Фигура 10. Абсолютно разположение, светла тема — контрола, при която разминаването би било изместване на всеки елемент поотделно.



Фигура 11. Правоъгълници с цвят, градиент и закръгление, тъмна тема — проверява цвят, геометрия и отсичане едновременно.



Фигура 12. Текстова контрола — проверява шрифт, разстояние между знаците и водещата линия едновременно.



Фигура 13. Списък с групиране — най-натовареният клас интерфейс: заглавия на групи, вътрешни отстъпи и превъртане.

Двойка ПРЕДИ и СЛЕД отстраняване на конкретно разминаване не се привежда като фигура: предишните състояния не са запазени като заснемания, а само като измерени стойности в дневника (например „6 px / 10,5 % → 0 / 0,5 %“). Числата са дадени при съответния клас в §6.3; изображенията не могат да бъдат възстановени, без съответната версия да бъде компилирана наново.

За анимираните интерфейси (37 на брой) фигурата би показвала *първия кадър*; където такъв се използва, това се отбелязва в легендата.

6.2.1. Най-слабо съвпадащите интерфейси

Прозрачността изисква да бъдат показани не само добрите случаи. Таблицата се генерира от данните; само колоната „причина“ се поддържа на ръка, защото оценката е измерване, а причината е находка — интерфейс без установена причина се отпечатва като „открит“, а не получава правдоподобно обяснение.

Таблица 8. Интерфейси със структурно сходство под 0,90

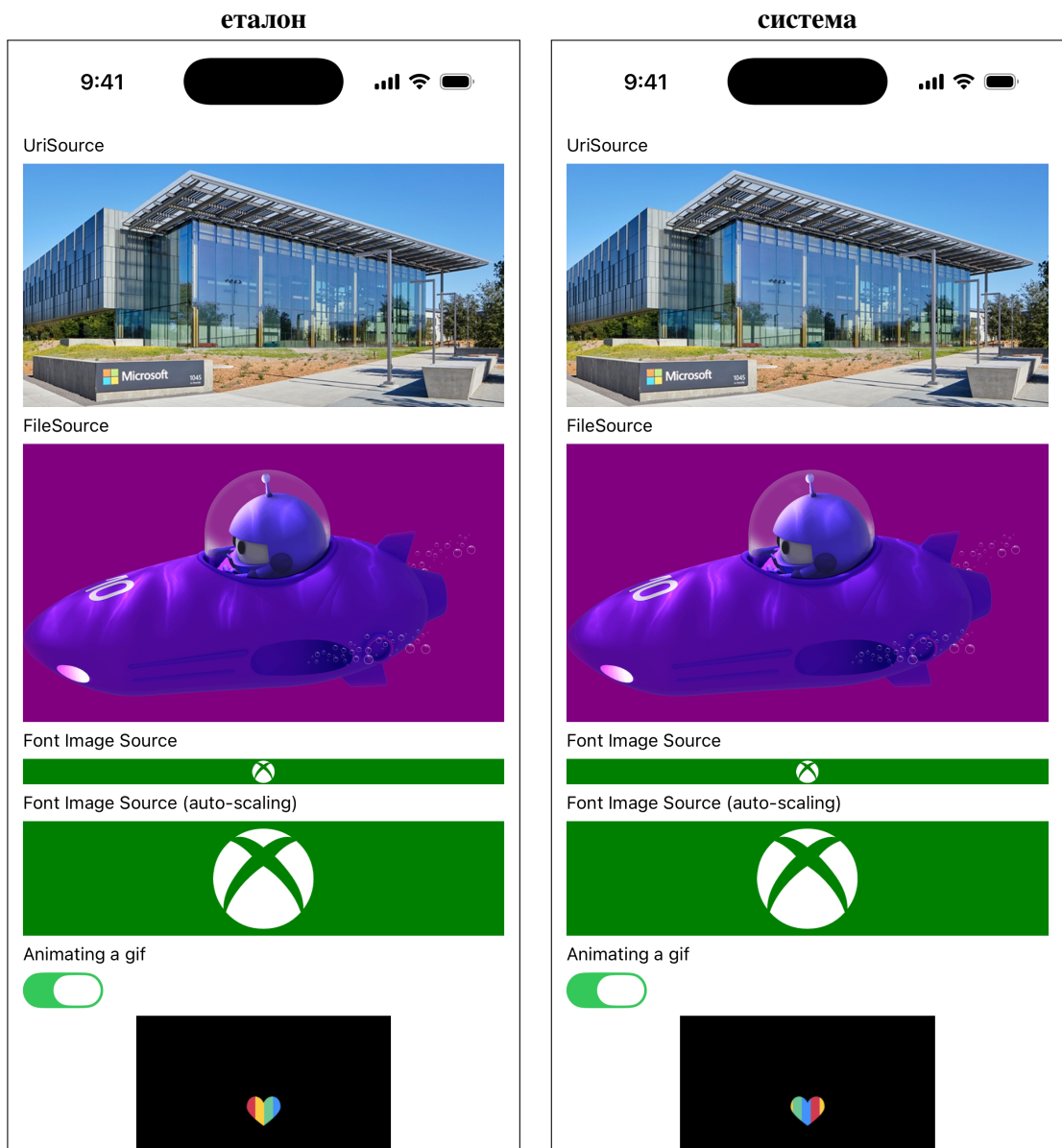
Интерфейс	Платформа	по код	от опис.	Причина
selection_	iOS	0,984	0,778	открит
synchronization				

Общо 1 двойки под 0,90, от които 0 под 0,75.

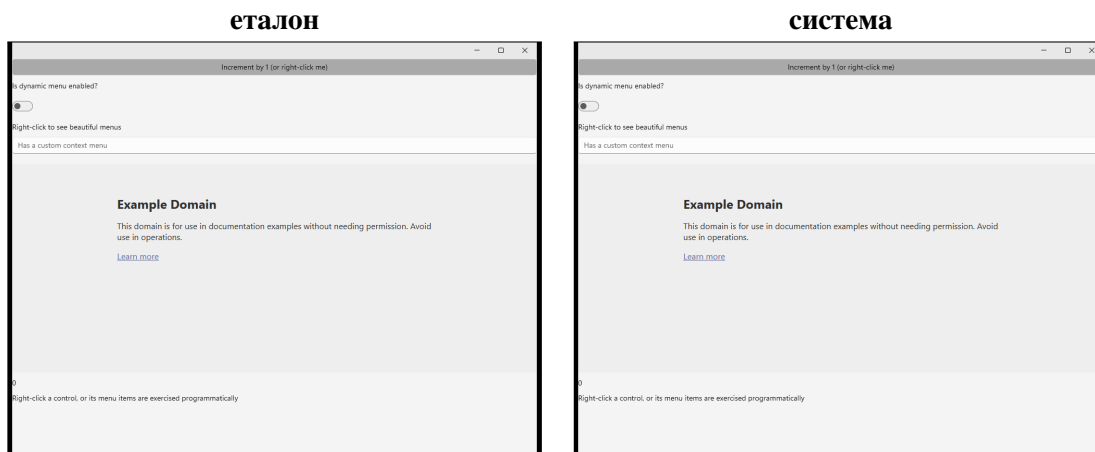
Към момента на писане таблицата съдържа един-единствен ред: същата двойка (iOS, входен път от описанието), която носи и единствената „съществена“ клетка в целия борд (§6.1). Разследването ѝ (§5.4.1) я класифицира като шум при кацане на анимация,

установен чрез контрола „еталонът срещу себе си“: повторни стартирания на самия еталон се разминават помежду си повече, отколкото системата с еталона. Колоната „причина“ отбелязва „открит“, не измислено обяснение: дали прагът за допустимо изместване при кацане да бъде вдигнат — или платформата изключена от изискването за възпроизводимо кацане — е оставено съзнателно отворено.

По-ранни разследвания оставиха и трайни методологични изводи. Интерфейсът `image` показва защо двата входни пътя не се събират в едно число: на Android само пътят по код се разминаваше съществено (0,478), докато пътят от описанието изобразяваше вярно (0,999) — осреднени, случаят би изчезнал безследно. Точно затова §5.3 изисква четирите сравнения да се отчитат поотделно, не като едно число на интерфейс.



Фигура 14. image на iOS — илюстрация на находката за двата входни пътя: в колоната на системата липсва само изтегляният по мрежата образ, останалите източници се изобразяват вярно.



Фигура 15. Интерфейс, изключен от количествената оценка по правилото за артефакт на измерването (§5.3): вгражда жива уеб страница, изтеглена от двете колони в различни моменти.

6.3. Систематизация на нарушенията на независимостта

Основният резултат на работата. Всяко разминаване, установено в хода на измерването и доведено до първопричина, попада в един от изброените по-долу класове. Систематизацията не е съставена по интуиция: тя обобщава *доведени до първопричина* случаи от кампаниите за отстраняване на разминавания, всеки документиран с измерване преди и след.

Общата констатация, която следва от разпределението им, е нетривиална и противоречи на очакването: *абстракцията не протича на границата на програмния интерфейс на контролите*. Съответствието „свойство → native свойство“ се пренася почти без съпротива. Протичането е на местата, където всеки инструментариум кодира *неизказана уговорка* — допускане, което не е част от документирания му интерфейс, но е част от поведението му.

Таблица 9. Класове нарушения на независимостта

№	Клас	Къде се проявява	Откриваем без екран?
1	допускания при наслагването	състав от слоеве	не
2	геометрични конвенции	измерване/подреждане	да
3	разрешаване на стойности и ред	таблици, стилове	да
4	входове от средата	всички платформи наведнъж	не
5	семантика на отстъпите	ръбове, вложени изгледи	частично
6	поведение само от реализацията	разклонения по тип	да
7	не са дефекти	—	—

Броят доведени до първопричина случаи във всеки клас не се привежда като число: той би отразявал реда, в който дефектите са били търсени, а не действителното им разпределение. Информативно е съотношението в последната колона — три от шестте действителни класа са откриваеми без екран, три изискват сравнение на изображения.

6.3.1. Клас 1 — допускания при наслагването

Разминавания, при които системата рисува вярно, но резултатът се наслагва върху основа с други свойства от очакваните — изолираното измерване на слоя показва правилен резултат, грешката се проявява едва в състава.

Разгледан пример — фон на прозореца, изграден от два слоя. Еталонът установява на прозореца системен ефект зад съдържанието (полупрозрачно замъгляване на работния плот), а върху него — *полупрозрачен* слой за областта на съдържанието. Наблюдаваният цвят на тялото е *съставен* от двете. Системата възпроизведе първо само първия слой: измереното — стойност 232 в светла тема — вече доказваше, че ефектът работи (непрозрачно запълване по подразбиране би дало 243), но цялото не съвпаднаше, защото липсваше вторият, полупрозрачен слой; добавянето на *непрозрачно* запълване на основата, обратно, се наслагва *пред* системния ефект и го обезсилва напълно. Правилното решение е основата да не се боядисва изобщо там, където ефектът е наличен.

Класът е поучителен по два начина: изолираното измерване на всеки слой поотделно дава правилен резултат — грешката е само в състава; и посоката на поправката е обратна на интуицията — не „добави още боя“, а „махни я, за да се вижда това, което е отдолу“. (Тъмната тема отчиташе яркост 32 при същото положение; заснемане отпреди

поправката не е запазено — бордът съхранява текущото състояние, не историята на изображенията — запазени са само измерените стойности.)

6.3.2. Клас 2 — геометрични конвенции

Класът с най-много случаи: всяка от двете страни има вътрешно съгласувана конвенция, конвенциите не съвпадат, и нито една не е записана в интерфейса.

Разгледан пример — несиметрично отчитане на външния отстъп. Договорът за измерване изисква измерващата фаза да *прибави* външния отстъп към съобщения размер, а подреждащата — да го *извади* от рамката; при нулев отстъп двете се компенсират, затова грешката е невидима в най-простия случай. В реализацията базовият изглед изпълняваше и двете, но преопределянето за контейнерите — само изваждането. Следствието е, че рамката на *всеки* контейнер губеше двукратния си отстъп: при поискана ширина 100 и отстъп 12 подреждането даваше 76 вместо 100.

Двете фази и посоката на прибавяне/изваждане са показани на (Фиг. 5).

Преди установяването на действителната причина бяха проверени и *отхвърлени с измерване* две правдоподобни хипотези — че вътрешният отстъп свива изричния размер, и че конкретна контрола пренебрегва собствения си външен отстъп. Отхвърлени чрез насочена проверка без екран, не чрез разсъждение, те са записани, за да не бъдат изпробвани отново.

Втори пример — половинпикселно свиване на контура. Инструментариумът за чертане свива контура на фигура с половин единица навътре, за да го центрира върху ръба. Същата фигура обаче служи и като *път на отсичане* за съдържанието вътре в нея, където свиването няма основание — води до отрязване на половин пиксел по целия периметър, дребно на всеки ръб и видимо като цвят по цялата рамка.

Трети пример — координатната система при подреждане (§3.5): абсолютни спрямо относителни на контейнера координати. Погрешният избор води до двойно отместване само при вложен контейнер с ненулево начало — не при най-простите интерфейси, типично за класа.

6.3.3. Клас 3 — разрешаване на стойности и ред на прилагане

Разгледан пример — зависимост от реда в таблицата за преобразуване. Размерът на бутон се изчислява от вътрешните отстъпи на native контролата, които трябва да включват дебелината на контура. Двете свойства се прилагат от таблицата в определен

ред — отстъп преди контур — така че при изчисляване на размера дебелината още е нула и контурът никога не достига до отстъпите. Еталонът оцелява при същия ред, защото на по-горното си ниво *замества* действието за отстъп с преизчисляване при измерване, тоест чете дебелината в момента на измерването: *редът на прилагане в таблицата е част от наблюдаваното поведение*, не вътрешна подробност. Решението е изчислението да получава дебелината като аргумент, независимо от реда — поправка на класа, не на случая.

Втори пример — минимална ширина, наложена от стила на платформата. С две присвоения еталонът занулява минималната ширина и височина на превключвател без текст. На една от платформите те са *безрезултатни*: действието за минимална ширина в таблицата изтрива локално зададената стойност, след което влиза минимумът от стила на инструментариума — 120 единици, надделяващ дори над изрично зададена от приложението ширина. Системата пренесе двата реда буквално и получи 32 единици вместо 120. Вярното пренасяне е *да не се пренасят*: незадаването е точно състоянието след изтриването, и минимумът от стила се проявява от само себе си — буквалното пренасяне на ред от еталона тук *разминава* поведението.

Трети пример — „незададено“ срещу „зададено на стойността по подразбиране“. Проверка, която пита за *стойността* вместо за *наличието* на стойност, не разграничава двата случая: контрола с изрично зададен черен цвят на текста се третира като без зададен цвят и получава цвета по подразбиране на платформата — бял при тъмна тема, текстът невидим. Поправката е изрична проверка „това свойство задавано ли е“, различна от сравнение със стойността.

6.3.4. Клас 4 — входи от средата

Разминавания, чийто източник е стойност, която приложението *не* задава, а получава от средата: проявяват се едновременно на всички платформи, а причината е на едно място.

Разгледан пример — източникът на темата. Приложението има слот за тема, но началната му стойност *не* е „светла“ — еталонът я установява в конструктора си от операционната система. Системата отначало пропускаше този един ред, при което слотът оставаше на подразбиращата се стойност: на машина с тъмна тема всяка платформа едновременно показва интерфейс, изграден по светли правила, върху тъмна системна основа. Клас, при който всяка платформа се проявява поотделно, почти никога не е от

този вид.

Интерфейсът, който свързва темата (`app_theme_binding`), е и този, на който разминаването се вижда най-пряко. Измереното след поправката: от 4,6 % различаващи се пиксела до 0,08 % — тоест от съществено разминаване до точно съвпадение. Заснемане отпреди поправката не е запазено по същата причина; запазено е измерването.

6.3.5. Клас 5 — семантика на отстъпите и безопасните зони

Класът, който най-ясно показва, че протичането не е между *платформи*, а между *уговорки* — две среди от едно и също семейство могат да се разминават помежду си.

Разгледан пример — двойно отчитане на безопасната зона. Отстъпът на безопасната зона се подава на съдържанието на страницата, но съдържанието вече е отместено от ръба и от собствения си външен отстъп. На мобилната среда системната лента *покрива* съдържанието — двете величини се мерят от един и същ ръб — така че подаването на пълния отстъп върху вече отместено съдържание го брои двукратно; поправката изважда собствения отстъп. Същата поправка обаче *влоши* настолната среда от същото семейство: там платформата на прозореца стои *над* областта на съдържанието, величините се сумират, и еталонът изобразява сбора им. Наложил се разклонение по среда — изваждане на едната, пълен отстъп на другата.

Втори пример от същия клас. Подравняването на съдържанието при списъчна контрола: еталонът установява стойност, при която съдържанието минава под системната лента. Пренасянето ѝ поправи осем интерфейса на мобилната среда и същевременно влоши настолната, където един и същ ред от изходния текст на еталона изобразява различно.

Изводът е по-силен от двата случая поотделно: *една и съща стойност в един и същ изходен текст на еталона поражда различно изобразяване в две среди от едно семейство*. Абстракция, която предполага, че семейството е еднородно, ще сгреша — и ще сгреша безшумно, защото на едната среда резултатът е правилен.

6.3.6. Клас 6 — поведение, изводимо само от реализацията на еталона

Клас, при който разминаването не се дължи на уговорка, а на поведение, което не следва от описанието и се открива само чрез прочитане на реализацията.

Разгледан пример. Заглавната и заключителната част на списъчна контрола се изобразяват или не в зависимост от *счетоманието* на вида разположение и дали елементите са групирани — вътрешното разпределение на еталона се разклонява по типа

разположение и при групиране излиза предсрочно, преди да достигне до тях. Системата беше приложила правилото на единия вид разположение върху всички; поведението не е документирано никъде и не следва от нито един тест, изведено е само от прочитане на разпределението по тип.

6.3.7. Клас 7 — разминавания, които не са дефекти

Два подкласа, разграничени изрично:

1. **Особеност на конкретен платформен инструментариум**, която еталонът също търпи — възпроизвеждането ѝ би било грешка.
2. **Артефакт на измерването** (§5.5, §6.1.1).

Класът съществува, защото и трите смушаващи фактора от §5.5 бяха първоначално диагностицирани като дефекти на системата — изброяването им предпазва следващия изследовател от същата загуба.

6.3.8. Изводи от систематизацията

Първо — къде да се насочи вниманието. Не към програмния интерфейс на контролите: съответствието „свойство → native свойство“ се пренася почти без съпротива. Вниманието принадлежи на конвенциите — геометрични, стойностни, средови — които никой инструментариум не документира, защото за него самия те са очевидни.

Второ — разделение на средствата за откриване. Класове 2, 3 и 6 са откриваеми без екран, за сметка на преносимата логика, и принадлежат на модулното тестване. Класове 1, 4 и 5 изискват сравнение на изображения, защото се проявяват само в състав или само на конкретна среда. Никое от двете средства не покрива другото.

Трето — следствие за реда на работа. Тъй като половината класове са невидими без сравнение на изображения, измервателният апарат трябва да се изгражда *успоредно* със системата, а не след нея: изграден след това, той открива наведнъж натрупани разминавания с вече преплетени причини; изграден успоредно, ги открива по едно, докато причината е още една.

6.4. Цена на реализацията

Измерванията тук са възможни само защото предходните установяват *контролно условие*: двете реализации изобразяват едно и също съдържание върху едни и същи native

контроли; без него разлика в размер или бързодействие не би била отличима от разлика в обхвата на реализираното.

Числата важат за платформи с потвърдено контролно условие, измерени с `measure_size.py` върху артефакти от 2026-08-16 и поправени за три дефекта в самия инструмент: несравними мерни състояния (на диск срещу отрязано), пропуснато изключване на директории, създадени при изпълнение, и избор на грешен главен файл при архивни формати. Единицата навсякъде по-долу е **MiB** (2^{20} B); стойности, публикувани по-рано с надпис „MB“, но вече изчислени в MiB, са преномерени само в надписа.

6.4.1. Размер на артефакта

Таблица 10. Размер на артефакта, отрязано срещу отрязано, release-компиляция от двете страни (MiB, 2026-08-16)

Платформа	еталон	система, по код	система, от опис.	съотношение (по код)
Mac Catalyst	47,29	9,14	22,13	5,17×
iOS (симулатор)	49,64	9,21	22,52	5,39×
Windows ¹	149,18	9,17	10,38	16,3×
Android ²	34,33	30,56	36,86	1,12× (портът по-малък)

¹ разгръщано състояние, без кеша на WebView2, записван при изпълнение (виж (г) по-долу). ² една процесорна архитектура, разопаковано, изцяло отрязано (виж (г) по-долу).

(а) Симетрия на обработката. Чете се отрязаното състояние, не суровият размер на диска: native артефактите се разпространяват без символи, затова диск и отрязано съвпадат, докато управляваният носи 31,78 MiB символна информация в изпълнимия файл (Mac Catalyst) — премахната само от едната страна дава 8,61× вместо 5,17×, завишение с 66 %.

(б) Разпределение по компоненти (Mac Catalyst). От 15,0 MiB метаданни и управлявани библиотеки на еталона — 8,53 MiB среда за изпълнение и базови библиотеки, 5,96 MiB код на фреймуорка, 0,49 MiB код на приложението: средата за изпълнение води, но фреймуоркът е **40 %** от разпределените байтове, не пренебрежим остатък. Преобладаващата компонента — 31,5 MiB предварително компилиран образ — остава **неразпределена**: маркерите в него са структури от данни, не граници на код, а разбор на формата не е направен.

(г) Три числа, публикувани по-рано, не издържаха повторна проверка — методологичен резултат сам по себе си, не недостатък на порта, докладван честно вместо

премълчан.

Android, „двойна архитектура“. По-раншното твърдение, че портът не извлича собствената си библиотека от APK-а, не се потвърждава: `extractNativeLibs` е `true` по подразбиране и манифестът на порта не го отменя, така че платформата извлича библиотеката въпреки STORED-опакването. Реалната асиметрия е друга — еталонният APK носи *две* процесорни архитектури (198 споделени библиотеки общо), портът — *една*; устройството зарежда само една, затова сравнение с втората еталонна архитектура вътре сравнява две приложения с едно. При еднакво условие съотношението за теглене се *обръща*: $0,63\times$, портът по-голям, а за заемано, отрязано място е практически наравно, $1,12\times$ ((Табл. 10)). Произходът е конфигурационен от двете страни, не свойство на езика: еталонният проект не декларира архитектура и изграждането взима подразбиращо и двете, докато портовият пресет избира изрично една.

Windows, *кеш на WebView2*. Публикуваното по-рано съотношение $4,7\times$ подценяваше предимството на порта: 74,6 % от измерената директория на порта е кеш, записан от *WebView2 при изпълнение*, не изграден артефакт. Изключен той ((Табл. 10)), съотношението е $16,3\times$ — най-голямото от четирите платформи, не най-малкото.

Windows, *различен модел на разгръщане*. Еталонът носи *цялата self-contained* среда `WindowsAppSDK` със себе си (135,18 MiB, 90,5 % от товара му); портът разчита на инсталираната на машината среда и носи само зареждащия компонент. Съотношението $16,3\times$ измерва модел на разгръщане, не „управляван срещу native“. Единственото еквивалентно сравнение — собствен код срещу собствен код — дава 4,99 срещу 4,97 MiB, практически равни (с уговорката, че IL е по-плътен от машинен код — равенство по байтове, не по обем реализация).

6.4.2. Защо декларативният вариант е по-обемен от кодовия

Двата варианта се различават съществено по размер ((Табл. 10)). Естественото предположение — разликата идва от вградените описания — е вярно само отчасти. Действителният обем на вградения markup е **0,35 MiB** (не 0,78 MiB, изчислявано по-рано — тази по-голяма стойност всъщност е *прирастът*, не обемът на самите описания).

Таблица 11. Състав на два native артефакта на Mac Catalyst (release, без символна информация, MiB)

Секция	по код	от описанието	разлика
__text (машинен код)	5,68 MiB	9,65 MiB	+3,97 MiB
__const (константни данни)	0,50 MiB	1,29 MiB	+0,79 MiB
__TEXT общо	7,28 MiB	14,34 MiB	+7,07 MiB

Секцията за низове `__cstring` се различава с под 20 KiB между двата варианта и не участва в анализа.

Markup-ът присъства в артефакта *двакратно* — веднъж като вграден байтов масив, веднъж като копие в константен низ, разбираан при първо извикване; удвоеният обем ($\approx 0,70$ MiB) обяснява 86–90 % от прираста на `__const` на трите платформи (macOS +0,79, iOS +0,81, Android +0,78 MiB). Прирастът на `__text` е около пет пъти по-голям — разликата е машинен код, не данни.

Причината следва от липсата на рефлексия (§2.6): кодовият вариант извиква само конструкторите на контролите, които реално ползва, а свързващият редактор изхвърля останалото; декларативният не знае предварително кой тип ще се появи и затова носи регистър на всички възможни типове — фабрика на обект и преобразувател „текст → стойност“ за всяко свойство — плюс разбора, обектния граф и разрешаването на свързванията. Всяка регистрация е код, недостъпен за изхвърляне, защото се достига по данни; управляваната среда плаща същото, но веднъж, защото механизмът ѝ за рефлексия вече е в средата за изпълнение.

На Android прирастът на кода е под една десета от Apple платформите (0,1 срещу $\approx 4,2$ MiB): портът там е споделена библиотека и всяка инстанциация на шаблон изнася манглирано име в таблицата на символите — прирастът се измества от код в изнесени символни имена. Причината за десеткратната разлика по мащаб не е установена докрай (оптимизацията и броят на страниците са отхвърлени).

6.4.3. Време до първи кадър (H2b-1)

Единствената от трите хипотези за бързодействие, за която има измерване, по определението, регистрирано предварително в §5.6: времето от стартиране до *първия заснет кадър, който не е начален екран* — изрично не до появата на прозорец. Всяка стойност е медиана от 10 студени стартирания.

Таблица 12. Време до първи кадър при студено стартиране (медиана; в скоби 95-и процентил)

Платформа	еталон	по код	от описанието
Android	3,615 s (4,882)	2,403 s (2,815)	2,381 s (2,697)
iOS	1,465 s (1,505)	1,264 s (1,271)	1,241 s (1,261)

Неизмерени платформи: **Mac Catalyst** — capture path is SSH -> screencapture -> scp (~1-3 s per sample); TTFF is ~1 s, so instrument resolution matches the signal; **AppKit** — same as maccatalyst — VM capture round trip dominates the measurement; **Windows** — app builds and runs on the guest; host-side polling measures transport, not startup.

Системата стартира по-бързо на двете измерими платформи: **1,42 пъти** на Android и **1,73 пъти** на iOS. Двата входни пътя се държат еднакво — декларативният не плаща за себе си при стартиране, защото разборът е извършен при компилация (§4.7).

Ограничение. Разделителната способност на измерването е интервалът на извадката — около 0,24 s. Разликите между двата входни пътя на системата (0,044 s на Android, 0,062 s на iOS) са *под* тази граница и не са разрешими — докладват се като „еднакви“; разликата спрямо еталона (0,43 s и 0,84 s) е няколкократно над границата и е разрешима. (По-ранен пробег на същия инструмент даде за Android $3,1\times$ вместо $1,42\times$ при непроменен код — разликата идва от състоянието на емулятора; цитира се последният пълен пробег.)

Три платформи остават неизмерени по причина, записана в самата таблица: заснемането там минава през виртуална машина и обиколката отнема от порядъка на самото измервано време.

6.4.4. Потребена памет и останалите хипотези за времето

Измерванията по **H2a** (потребена памет), **H2b-2** (разположение) и **H2b-3** (установен режим) *не са проведени* към момента на предаване. H2b-3 е формулирана *срещу* очакването на автора — прогнозира липса на разлика при установен режим; потвърждаването на H2b-1 не е довод в нейна полза, защото двете измерват различни неща.

6.4.5. Съответствие с предварително регистрираните хипотези

Таблица 13. Съответствие между регистрираните хипотези и измереното

№	Прогноза (регистрирана предварително)	Изход
H1	управляваният артефакт е многократно по-обемн; разликата се дължи специфично на средата за изпълнение	частично потвърдена — посоката се потвърждава навсякъде; величината варира силно: $5,17\text{--}5,39\times$ на Apple, $16,3\times$ на Windows (собствен код $1,00\times$), Android практически наравно по заемано място, по-голям за теглене ($0,63\times$)
H1'	разликата е голяма на Apple платформите и малка на Windows, където и двете колони изискват една и съща платформена среда	частично потвърдена — вярна само за собствения код ($1,00\times$); еталонът на Windows е self-contained, не споделя средата, затова пълното съотношение е <i>най-голямото</i> от всички ($16,3\times$)
H2a	по-малка следа на паметта	непроверена
H2b-1	значително по-бързо студено стартиране	потвърдена — $1,42\times$ на Android, $1,73\times$ на iOS
H2b-2	измеримо предимство при разположението	непроверена
H2b-3	<i>липса</i> на разлика при установен режим	непроверена

Три от шестте прогнози остават непроверени, таблицата го казва вместо да го премълчи. Ползата от предварителната регистрация личи при H1 и H1': без нея формулировките биха били пренаписани мълчешком след намирането на трите методологични дефекта в (г) по-горе; регистрираният текст остава свидетелство какво точно е било предвидено, преди да е било измерено.

6.5. Ограничения на изследването

- Обхват на проверката за достоверност.** Механичната проверка (§5.5) улавя размножен кадър и неприложена тема, но не улавя кадър, заснет по време на преход, ако размерите му са правилни. Такъв кадър би понижил оценката, тоест грешката е в консервативната посока — но остава неоткривана автоматично.
- Освобождаване на памет.** Изолацията на всяко измерване (§5.2) прави невъзможно наблюдението на освобождаване при напускане на интерфейс — измерва се следа, не

своевременност.

3. **Разделителна способност при измерване на стартиране** — определението стъпва на честотата на заснемане и не претендира за милисекундна точност.
4. **Обвързаност с една версия на еталона и с едно измерване на размера.** Сравнението за степен на съвпадение е спрямо конкретна разпространена версия; числата за размера са от едно измерване (2026-08-16) и не се пренасят автоматично към следваща версия на нито едната страна.
5. **Подбор на интерфейсите.** Наборът е съставен за покритие на контролите и алгоритмите, не като извадка от типични приложения — заключенията се отнасят за покритието, не за средното приложение.

ЗАКЛЮЧЕНИЕ

В хода на дипломното проектиране е изградена система, която описва графичен потребителски интерфейс независимо от платформения инструментариум, и — по-съществено — методика, с която описанието може да бъде *проверено*, а не само заявено. По задачите от Увода: преглед на подходите показва защо твърдението за еднакво изобразяване остава непроверено (Раздел I); алтернативите са анализирани по пет предварително изброени критерия, с избор, обоснован чрез преброяване в кодовата база, не преценка (Раздел II); проектирана е слоеста архитектура, при която платформено обусловената част е сведена до таблично зададено съответствие (Раздел III), реализирана върху четири native инструментариума и един слой без екран (Раздел IV); изградена е методика за автоматизирано измерване на еквивалентността на изобразяването (Раздел V) и е приложена върху 172 интерфейса, откъдето е изведена систематизация на установените разминавания (Раздел VI).

Приноси.

1. **Систематизация на нарушенията на независимостта** — шест класа, изведени от случаи, доведени до първопричина: абстракцията не се нарушава на границата на програмния интерфейс, а там, където инструментариумът кодира неизказана уговорка — констатация, обобщима извън конкретния артефакт.
2. **Методика за количествено измерване на еквивалентността** между две независими реализации на едно описание — правилата за оценяване, записани предварително, и механичните проверки за достоверност на измерването.
3. **Разграничение на средствата за откриване:** половината класове са откриваеми без екран, половината изискват сравнение на изображения — затова измервателният апарат трябва да се изгражда успоредно със системата, не след нея.
4. **Реализацията като инструмент** — система върху пет платформени слоя, която прави измерването изобщо възможно, с публичен интерфейс, при който най-честият клас грешки при управление на паметта е грешка при компилация.
5. **Предварително регистрирани хипотези** за цената на реализацията, с изрични изисквания за валидност — включително хипотеза, формулирана срещу очакването на автора.

Предимства на предложеното решение. Спрямо алтернативите от Раздел II решението печели по трите критерия, определящи изхода на работата. По **K1 (проверимост)** — послойното изграждане, водено от тестове, е единствената стратегия с общо описание, изпълнявано от двете реализации, тоест с контролно условие за сравнение. По **K4 (безопасност по конструкция)** — само преместваемият притежаващ манипулатор превръща най-честия начин за създаване на цикъл в грешка при компилация, с измерена ергономична цена само в малцинство от местата. По **K5 (цена за платформа)** — изборът на език дава пряк достъп до три от петте инструментариума без нито един ред обвързващ код.

Отделно стои предимство извън първоначалната цел: на настолната Apple платформа еталонът предлага *само* мобилния инструментариум, пренесен на настолна машина, докато разработената система носи *и двата* слоя — същия мобилен път за прякото сравнение в Раздел VI, и втори слой върху native настолния инструментариум с контроли на самата операционна система, без промяна в описанието на интерфейса. Щом платформената част е сведена до таблица от действия, две различни таблици за една операционна система са обикновен случай, не изключение.

Печели се и нещо друго, също непредвидено: побайтово съвпадение с еталона за част от изображенията (§6.1.1) — най-силната форма, която резултатът от такова измерване изобщо може да приеме.

Недостатъци и ограничения.

1. **Четири от шестте регистрирани хипотези остават непроверени.** Измерванията за потребена памет и за време за реакция не са проведени — инструментът е проектиран, но не изпълнен. Единствената проверена хипотеза се оказва *твърде силна* по величина.
2. **Паритетът е практически, но не напълно постигнат.** По код бордът е свободен от съществени разминавания на всичките 172 страници и четирите платформи (Табл. 6); незначителните остават 17. Четенето от описанието добавя един съществен случай (iOS, *selection_synchronization*) (Табл. 8) — разследван до първопричина и определен като шум при кацане на анимация, не дефект на порта (клас 7 в §6.3); цветът остава непроменен, защото прагът за приемане на такъв шум е оставен отворен.
3. **Един платформен слой е реализиран частично по замисъл** — изобразяват се само част от контролите, останалите използват огледалото без екран.

4. **Цената на модела на собствеността е реална** — достъп до задържана контрола изисква проверка за живост, съвместното притежаване — изрично действие: съзнателна размяна на удобство срещу проверимост, не безплатно предимство.
5. **Инструментът не може да наблюдава освобождаване на памет** при напускане на интерфейс, тъй като всяко измерване се извършва в отделен процес (§5.2) — ограничение на конструкцията, не пропуск в изпълнението.
6. **Резултатът е обвързан с една версия на еталона** и не се пренася автоматично към следваща.
7. **Един критерий за достоверност остаря заради успеха на системата** (§6.1.1) — проверката обявява побайтовото съвпадение за дефект, вярно само докато то е било недостижимо.

Насоки за по-нататъшна работа.

1. **Провеждане на измерванията по H2a и H2b** — пряко следствие от ограничение 1. Инструментът е проектиран; остава изпълнението и отчитането, включително на отрицателния резултат, ако H2b-3 се потвърди.
2. **Стесняване на критерия за достоверност** — следствие от ограничение 7: признакът трябва да стане „междуклонно съвпадение *и* съвпадение между темите“, иначе броят лъжливи тревоги ще расте с качеството на реализацията.
3. **Довършване на частичния платформен слой** (ограничение 3) и повторно измерване — проверка дали систематизацията от §6.3 предсказва кои класове ще се появят на нов инструментариум.
4. **Платформен слой за Linux** — най-информативното продължение, защото еталонът *няма* такъв: той не рисува сам, а преобразува към native инструментариум, а на Linux няма *един* очевиден избор (GTK и Qt съжителстват). Разработената система няма това ограничение, но губи друго: без еталон на платформата отпада оракулът, а с него и методиката от Раздел V — проверката би останала вътрешна: съгласуваност между входните пътища и изчисленото разположение, без съвпадение с външен образец. Границата на самия метод, която новата платформа би направила видима.
5. **Разширяване на систематизацията с шести платформен слой** от друго семейство — всяка нова среда повтаря експеримента и проверява дали класовете са свойство на

задачата или на конкретните четири инструментариума.

- 6. Автоматизирано следене на разминаванията във времето** — запазване на заснеманията при всяка кампания, не само на текущото състояние: за кривата от §6.1 и липсващите съпоставки ПРЕДИ/СЛЕД (§6.2).

ЛИТЕРАТУРА

- [1] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, “The oracle problem in software testing: A survey,” *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.
- [2] A. Biørn-Hansen, C. Rieger, T.-M. Grønli, T. A. Majchrzak, and G. Ghinea, “An empirical investigation of performance overhead in cross-platform mobile development frameworks,” *Empirical Software Engineering*, vol. 25, no. 4, pp. 2997–3040, 2020.
- [3] Microsoft, “.NET multi-platform app UI (.NET MAUI) documentation.” <https://learn.microsoft.com/dotnet/maui/>. посетен на 4 август 2026 г.
- [4] Microsoft, “dotnet/maui — .NET MAUI source repository.” <https://github.com/dotnet/maui>. посетен на 4 август 2026 г.
- [5] Google, “Flutter documentation.” <https://docs.flutter.dev/>. посетен на 4 август 2026 г.
- [6] The Qt Company, “Qt documentation.” <https://doc.qt.io/>. посетен на 4 август 2026 г.
- [7] AvaloniaUI, “Avalonia UI documentation.” <https://docs.avaloniaui.net/>. посетен на 4 август 2026 г.
- [8] Meta Platforms, “React Native documentation.” <https://reactnative.dev/docs/getting-started>. посетен на 4 август 2026 г.
- [9] Microsoft, “Handlers — .NET MAUI.” <https://learn.microsoft.com/dotnet/maui/user-interface/handlers/>. посетен на 4 август 2026 г.
- [10] Apple, “UIKit — apple developer documentation.” <https://developer.apple.com/documentation/uikit>. посетен на 4 август 2026 г.
- [11] Google, “Views — android developers.” <https://developer.android.com/reference/android/view/View>. посетен на 4 август 2026 г.
- [12] Microsoft, “WinUI 3 — windows app SDK.” <https://learn.microsoft.com/windows/apps/winui/winui3/>. посетен на 4 август 2026 г.

- [13] V. Garousi, İ. B. Işık, A. Z. Boyraz, W. Afzal, B. Baydan, B. Yolaçan, A. Çağlar, S. Çaylak, and K. Herkiloğlu, “Comparing automated visual GUI testing tools: an industrial case study,” in *Proceedings of the 8th ACM SIGSOFT International Workshop on Automated Software Testing (A-TEST)*, (Paderborn, Germany), pp. 21–28, 2017.
- [14] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, “Image quality assessment: From error visibility to structural similarity,” *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.
- [15] N. A. Ernst and M. T. Baldassarre, “Registered reports in software engineering,” *Empirical Software Engineering*, vol. 28, no. 2, p. 55, 2023.
- [16] International Organization for Standardization, “ISO/IEC 14882:2024 — programming languages — C++.” ISO, Geneva, Switzerland, 2024.
- [17] cppreference.com, “C++ reference.” <https://en.cppreference.com/w/cpp>. посетен на 4 август 2026 г.

ПРИЛОЖЕНИЯ

Приложение А. Пълен изходен текст на срез Button

Срезът за бутон в пълен вид — договорът, обработчикът с двете си таблици и платформената реализация за една среда. В Раздел IV са дадени само откъсите, носещи проектните решения (листинги 1 и 2); тук се привежда цялото, за да е проверимо, че между откъса и цялото няма разлика.

Показани са само *относимите* откъси, а не целите файлове: пълните текстове са в хранилището и възпроизвеждането им тук би добавило десетки страници без да добави информация. Всеки откъс се *чете* от истинския файл при компилация (\lstinputlisting с диапазон от редове), тоест не може да се разминее с изходния текст — но и не го дублира.

```
1  class i_button : public i_view, public i_padding, public i_button_stroke
2  {
3  public:
4      // Called by the platform view on touch-down.
5      virtual void send_pressed() = 0;
6      // Called by the platform view on touch-up (inside or outside) and on cancel.
7      virtual void send_released() = 0;
8      // Called by the platform view on a completed tap (touch-up inside).
9      virtual void send_clicked() = 0;
10 };
11 } // namespace maui::core
```

Листинг 8: Договор — i_button: само свойства и събития, без изобразяване

```
1  void update_background(const maui::graphics::paint* value) override;
2  void update_transform(const maui::core::transform_spec& value) override;
3  void update_flow_direction(maui::core::flow_direction value) override;
4  void update_semantics(const maui::core::semantics* value) override;
5  void update_clip(const maui::graphics::i_shape* value) override;
6  // The clip borrow platform_arrange re-resolves against the live bounds (null = no clip)
↪ . Unused on
7  // headless/Apple (Apple stashes via an associated object), so it is android-gated.
8  const maui::graphics::i_shape* clip_shape = nullptr;
9  // The trampoline the installed View.OnClickListener carries as its peer, and which OWNS
↪ the
10 // send_clicked body. Heap-allocated and registry-registered so a tap that arrives after
↪ the
11 // handler is gone resolves to nothing instead of dereferencing freed storage --- the
↪ reason this
```

```

12      // is a shared_ptr rather than the 'this' the Java side used to carry as a raw jlong.
↳ Torn down by
13      // release_dialog_seam from BOTH on_disconnect_handler and ~button_platform.
14      std::shared_ptr<maui::platform::android::dialog_trampoline> dialog_peer;
15  #endif
16  };
17
18  class button_handler : public view_handler<button_handler, i_button, button_platform>
19  {
20  public:
21      button_handler();
22
23      // Shared mapper tables (cross-platform --- defined in button_handler.cpp). 'mapper'
↳ chains the
24      // text mapper + the image mapper, mirroring ButtonHandler.Mapper chaining
↳ TextButtonMapper +

```

Листинг 9: Обявяване на обработчика — трите таблици и създаването на native контролата

```

1  property_mapper<i_button, button_handler>& button_handler::mapper()
2  {
3      static property_mapper<i_button, button_handler> table = [] {
4          property_mapper<i_button, button_handler> mapped{
5              {"padding", &button_handler::map_padding},
6              {"stroke_color", &button_handler::map_stroke_color},
7              {"stroke_thickness", &button_handler::map_stroke_thickness},
8              {"corner_radius", &button_handler::map_corner_radius},
9          };
10         // Reverse-order iteration in keys() means the LAST chained mapper's keys come first
↳ , so listing
11         // image_mapper then text_mapper then view_mapper yields: view (generic IView) keys,
↳ then text
12         // keys, then image keys (C# lists TextButtonMapper, ImageButtonMapper, ViewMapper
↳ --- same set).
13         mapped.set_chained({&image_mapper(), &text_mapper(), &view_mapper()});
14         return mapped;
15     }();
16     return table;
17 }
18
19 // Cross-platform source routing --- image_handler::map_source's twin over the button
↳ platform primitives
20 // (ButtonHandler.MapImageSource → MapImageSourceAsync → ImageSourceLoader.
↳ UpdateImageSourceAsync): the
21 // file fast-path is synchronous; every other source (uri/stream/font) goes through the
↳ handler-owned
22 // loader (async, with the source-identity recheck); a null/empty source cancels + clears.
23 void button_handler::map_image_source(button_handler& handler, i_text_button& view)

```

```

24     {
25         auto* platform = handler.typed_platform_view();
26         if (platform == nullptr)
27         {
28             return;
29         }
30         // Record the current ContentLayout (position + spacing) on the mirror BEFORE the load,
↪ so the
31         // per-backend icon primitive (android apply_button_icon) places the compound drawable
↪ correctly ---
32         // "source" runs before "content_layout" in the mapper table, so the icon primitive can'
↪ t rely on

```

Листинг 10: Преносима реализация — веригата от таблици

```

1         // ButtonHandler.SetControlPropertiesFromProxy: adopt any UIAppearance-proxy values for
↪ the
2         // three control states up front ("If new values are null, old values are preserved" ---
↪ a nil
3         // proxy value leaves the UIButton's own default in place).
4         void set_control_properties_from_proxy(UIButton* button)
5         {
6             UIButton* const proxy = [UIButton appearance];
7             for (const UIControlState state : k_control_states)
8             {
9                 [button setTitleColor:[proxy titleLabelColorForState:state] forState:state];
10                [button setTitleShadowColor:[proxy titleLabelShadowColorForState:state] forState:
↪ state];
11                [button setBackgroundImage:[proxy backgroundImageForState:state] forState:state
↪ ];
12            }
13        }
14
15        // Rebuild the UIButton's attributed title from its current plain title, mirroring
16        // ButtonExtensions.UpdateCharacterSpacing (UILabel.AttributedText.
↪ WithCharacterSpacing +
17        // WithTextColor → SetAttributedTitle(..., Normal)). Kerning needs an attributed title,
↪ and a
18        // kerned title must carry its color explicitly (setTitleColor:/tintColor do not apply
↪ to an
19        // attributed title), so when spacing != 0 the title is rebuilt attributed WITH the text
↪ color.
20        // When spacing == 0 the attributed title is reset to nil so the plain title (colored by
21        // map_text_color) shows --- matching C#, where WithCharacterSpacing returns null on a
↪ fresh,
22        // un-kerned title and SetAttributedTitle(null) keeps the plain-title path.
23        void refresh_button_title_formatting(UIButton* button, const i_text_button& view)

```

Приложение Б. Публичен програмен интерфейс на системата

Публичните типове по слоеве, съпоставени с типовете на еталона. Съответствието е едно към едно по покритие; имената се преобразуват по правилата на езиковия профил, а всеки заглавен файл носи в коментар пълното име на съответстващия тип. Списъкът е генериран от първия ред на всеки публичен заглавен файл, който носи съответствието в коментар (`tools/gen_tex.py`), и обхваща **608** типа в 19 слоя — от `core` и `controls` до `layouts`, `graphics`, `hosting`, `bindings` и `xaml`.

Пълните 608 реда не се препечатват тук — добавят над 20 страници без да добавят информация отвъд самия списък. Табл. 14 показва ръчно подбрана извадка от основните слоеве, представителна за формата и покритието на цялата таблица.

Таблица 14. Представителна извадка от публичния програмен интерфейс (12 от 608 типа)

Слой	Система	Еталон
core	<code>maui::core::i_element_handler</code>	<code>Microsoft.Maui.IElementHandler</code>
core	<code>maui::core::i_layout_handler</code>	<code>Microsoft.Maui.ILayoutHandler</code>
core	<code>maui::core::button_handler</code>	<code>Microsoft.Maui.Handlers.ButtonHandler</code>
controls	<code>maui::controls::button</code>	<code>Microsoft.Maui.Controls.Button (M2 subset)</code>
controls	<code>maui::controls::border</code>	<code>Microsoft.Maui.Controls.Border</code>
controls	<code>maui::controls::absolute_layout</code>	<code>Microsoft.Maui.Controls.AbsoluteLayout</code>
layouts	<code>maui::layouts::layout_manager</code>	<code>Microsoft.Maui.Layouts.LayoutManager</code>
layouts	<code>maui::layouts::grid_layout_manager</code>	<code>Microsoft.Maui.Layouts.GridLayoutManager</code>
graphics	<code>maui::graphics::color</code>	<code>Microsoft.Maui.Graphics.Color</code>
hosting	<code>maui::hosting::maui_app_builder</code>	<code>Microsoft.Maui.Hosting.MauiAppBuilder</code>
bindings	<code>maui::controls::binding</code>	<code>Microsoft.Maui.Controls.Binding</code>
xaml	<code>maui::xaml::i_markup_extension</code>	<code>Microsoft.Maui.Controls.Xaml.IMarkupExtension</code>

Пълният списък от 608 типа е генериран автоматично от изходния текст — от първия ред на всеки публичен заглавен файл (`tools/gen_tex.py`) — и е достъпен в

хранилището на ангажимент a1d6dd9956: заглавните файлове са на <https://github.com/Alex-Tsvetanov/maui/tree/a1d6dd9956fa39126a094dcc43545b6beae5ef93/port/cpp/include/maui>, а генераторът — на https://github.com/Alex-Tsvetanov/maui/blob/a1d6dd9956fa39126a094dcc43545b6beae5ef93/port/cpp/docs/thesis/tools/gen_tex.py.

Приложение В. Съвпадение по интерфейси

Всеки от 172 интерфейса се оценява на четирите платформи по структурно сходство (§5.4) и попада в едно от три нива: съвпада, различава се незначително или се различава съществено. (Табл. 15) обобщава разпределението по платформа.

Таблица 15. Разпределение на оценките по платформа

Платформа	Съвпада (зелено)	Незначително (жълто)	Съществено (червено)
iOS	168	4	0
Mac Catalyst	169	3	0
Android	165	7	0
Windows	170	2	0

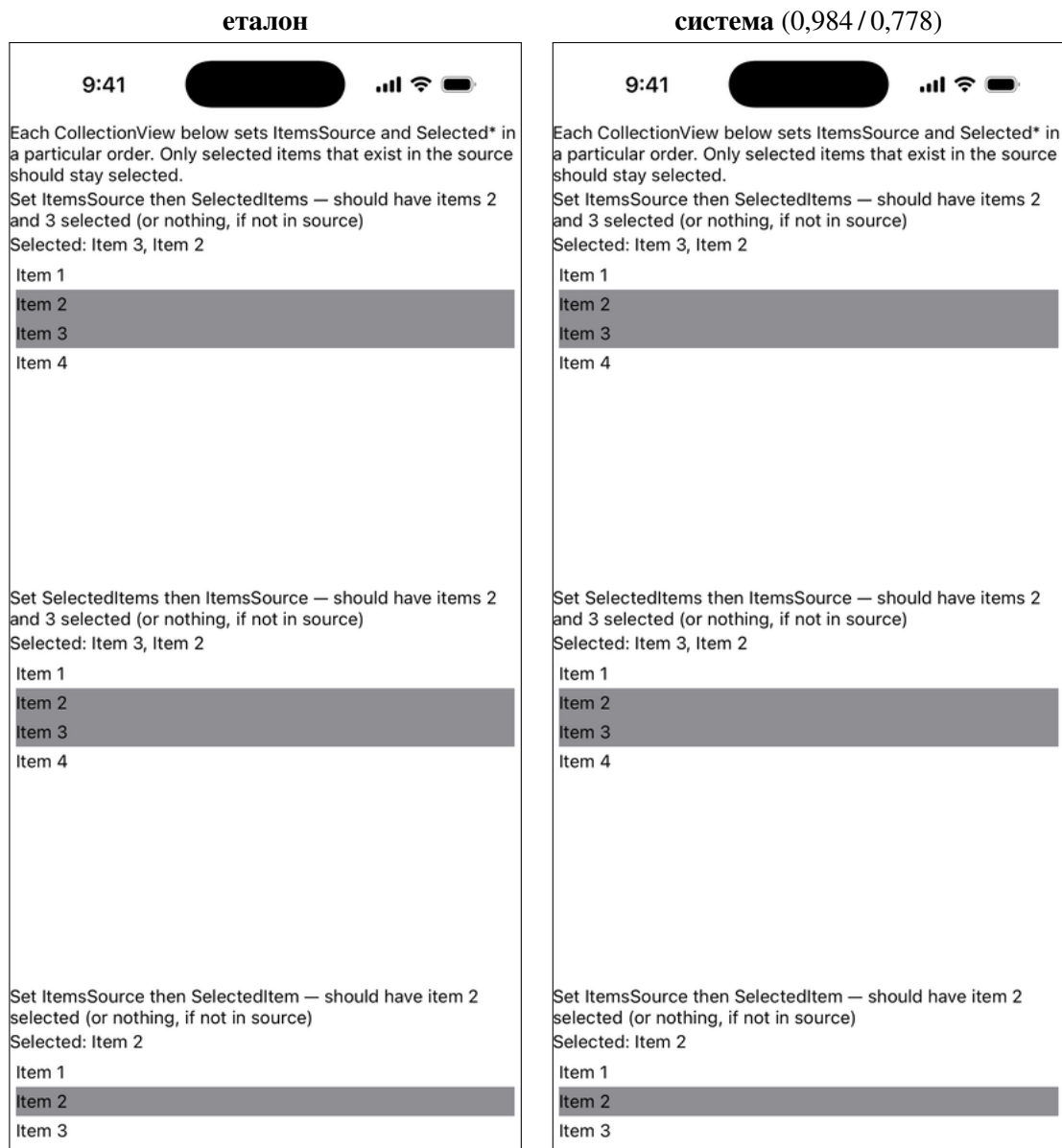
Таблицата показва входния път *по код*: по него няма нито един интерфейс със съществено разминаване на нито една от четирите платформи. По входния път *от описанието* остава едно изключение — iOS, `selection_synchronization` — разгледано в §6.2 ((Табл. 8)) заедно с констатацията защо не е принудено до зелено.

Разбивката по **отделен интерфейс** — каквато носеше предишният вид на тази таблица, ред по ред — не се препечатва тук: строи се възпроизводимо от `comparison.json` (`tools/gen_tex.py`). Изходните данни за всеки от 172 интерфейса, по двата входни пътя (код / описание), са версионирани на същия ангажимент: <https://github.com/Alex-Tsvetanov/maui/blob/a1d6dd9956fa39126a094dcc43545b6beae5ef93/port/cpp/docs/comparison/comparison.json>; четим изглед със статус по интерфейс и платформа — в <https://github.com/Alex-Tsvetanov/maui/blob/a1d6dd9956fa39126a094dcc43545b6beae5ef93/port/cpp/docs/comparison/README.md>.

Най-слабо съвпадащите интерфейси — съпоставки

Всеки интерфейс от (Табл. 8) е показан тук един до друг с еталона, в темата, която дърпа оценката надолу — показването на светлия кадър за разминаване, което съществува само в тъмната тема, не илюстрира нищо. Измерените стойности стоят в

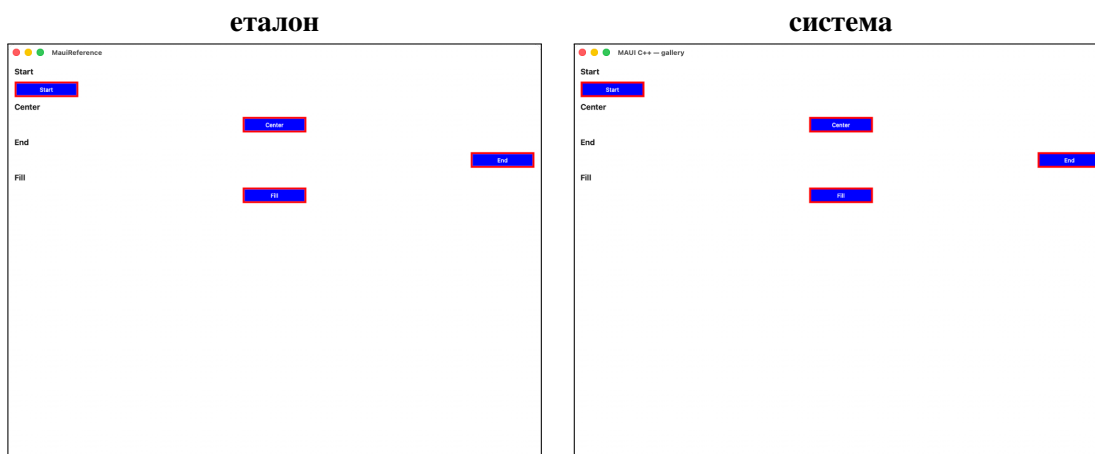
заглавието на дясната колона и в легендата, така че твърдението в текста може да бъде сверено с изображението, вместо да се приема на доверие.



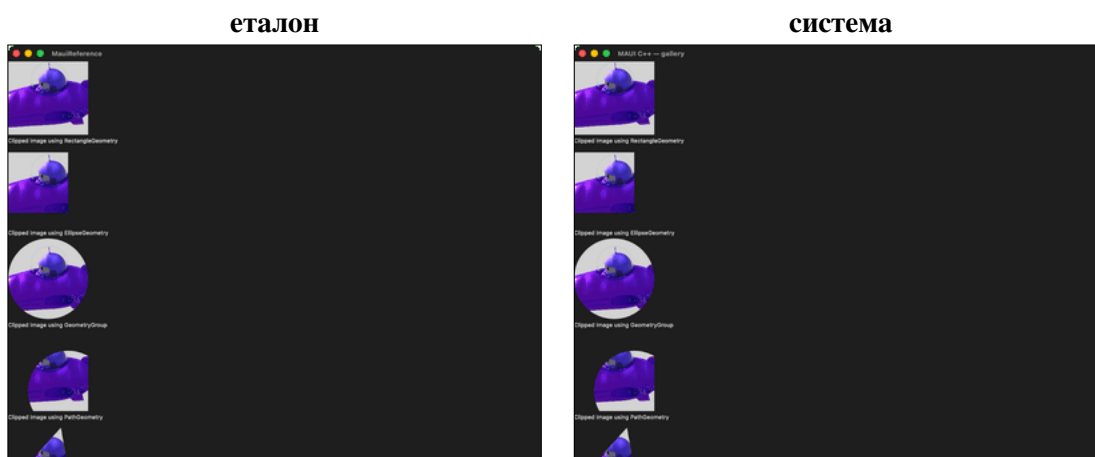
Фигура 16. selection_synchronization — iOS, светла тема. Структурно сходство 0,984 по код и 0,778 от описанието. Открит.

Приложение Г. Екранни форми

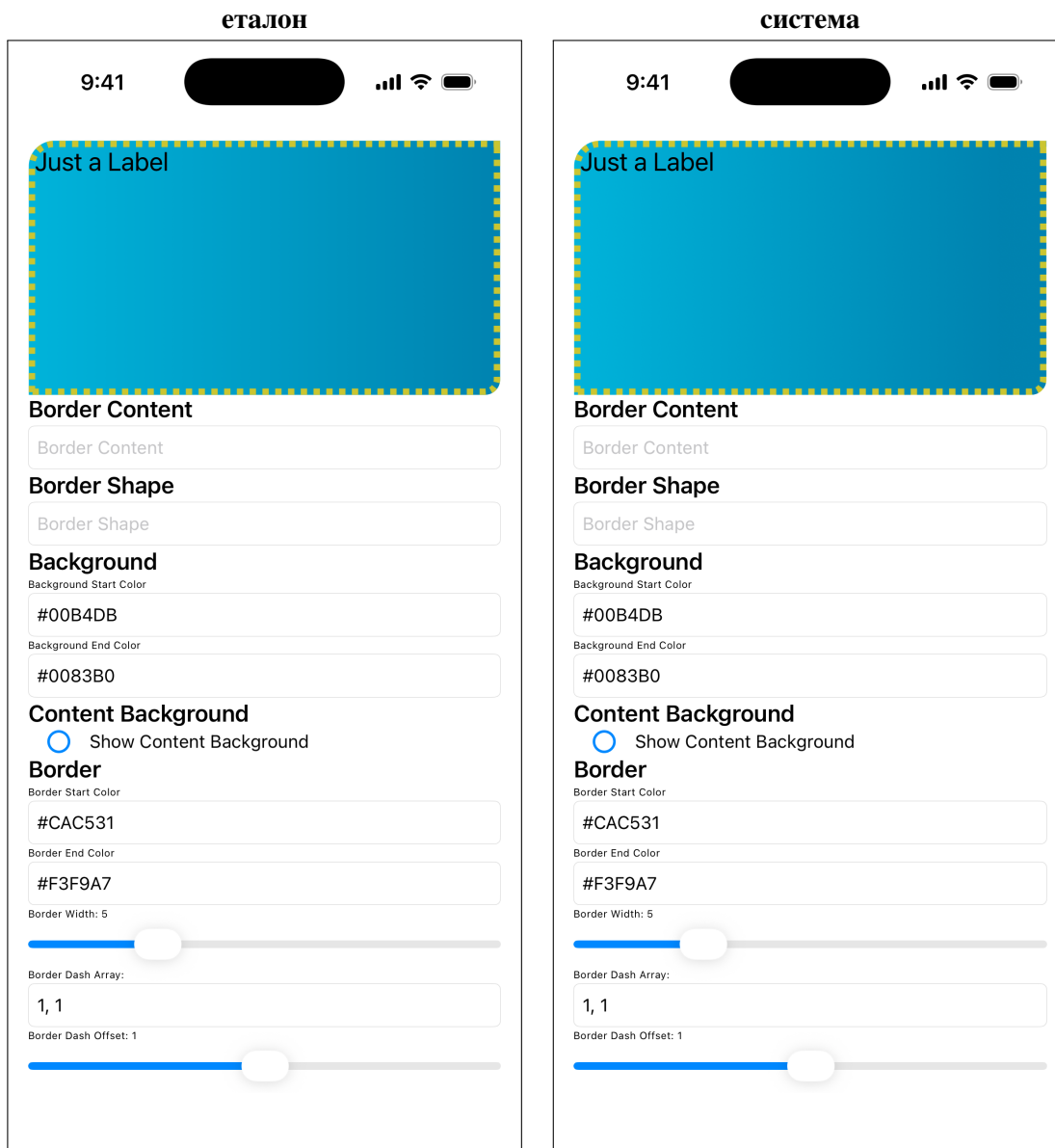
Съпоставки за интерфейси, които не са включени в Раздел VI. Всички са символни връзки към заснеманията от борда, тоест същите файлове, които са били оценени.



Фигура 17. Подравняване — проверява разположението без собствено изобразяване на контролите.



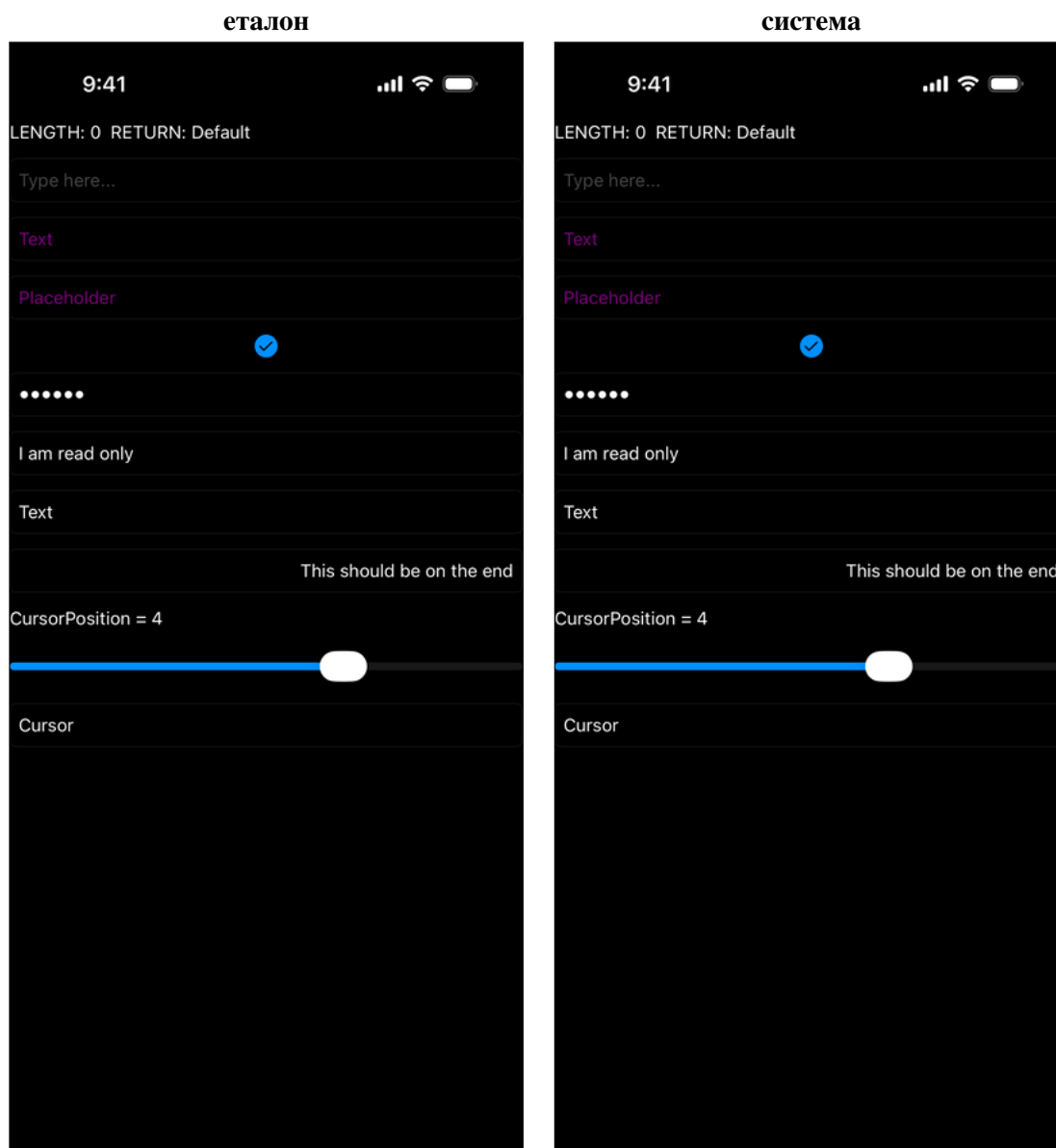
Фигура 18. Отсичане, тъмна тема — класът, при който половинпикселното свиване на контура се проявява (§6.3).



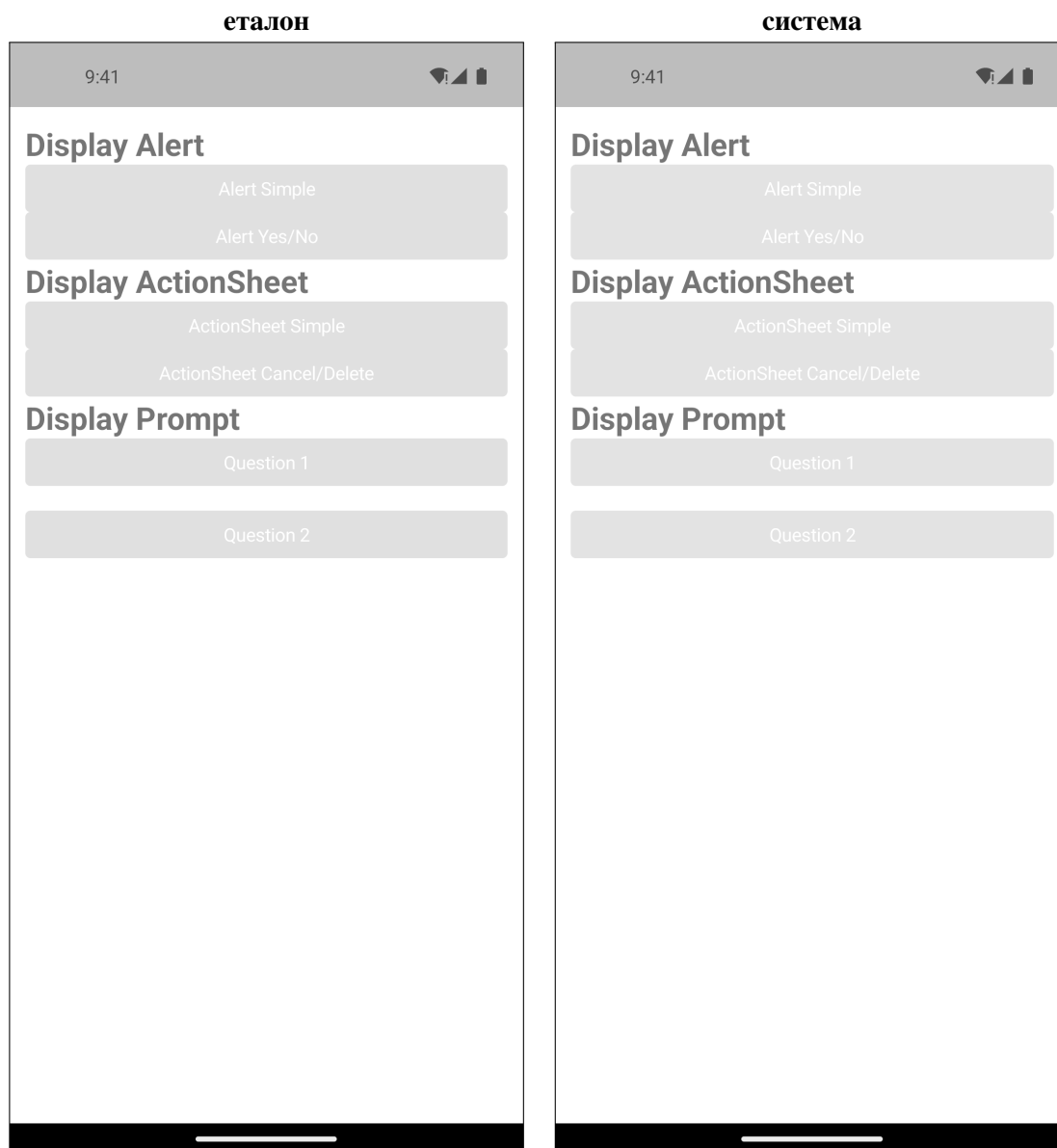
Фигура 19. Рамки и радиуси — интерфейст, върху който е измерена промяната в размера на превключвателя (§2.3).



Фигура 20. Решетка — распределяне на пропорционални измерения.



Фигура 21. Поле за въвеждане, тъмна тема — интерфейсьт, върху който се прояви размерът на превключвателя (§2.3).



Фигура 22. Съобщения — native диалози на трета среда.

За анимираните интерфейси (37 на брой) съпоставката би показала първи кадър; тук такива не са включени.